

3-其他

求职答疑&简历修改&offer选择

求职答疑

面试中怎么保持思考

简历上的项目怎么选&组合

简历修改

安全背景的Agent简历

大模型算法背景

有算法背景，但主线发散

bg不错大厂约面少，字节横向，腾讯一面挂

offer选择

个人面经

暑期

阿里暑期一面面经

美团暑期二面面经

美团暑期一面面经

字节暑期二面面经

字节暑期三面面经

字节暑期一面面经

个人整理项目笔记

后端

点评

点评

大营销

分库分表

简历撰写参考

具体业务

设计模式

数据结构

[消息队列](#)

[压测与瓶颈分析](#)

[DDD](#)

[redis](#)

[agent](#)

[nanobot](#)

经验分享

[Java后端开发求职分享-我的实习_秋招复盘](#)

[点评_外卖之类的“烂大街”项目还值得做吗](#)

[三个月内做完一个完整项目-面向面试学习](#)

[小厂实习也可以是加分项](#)

[校招大厂后端面试，到底怎么准备场景题](#)

[叙述项目_场景题的回答模板](#)

求职答疑&简历修改&offer选择

面试中怎么保持思考

总结

问题：

面试能力怎么快速补；

面试中怎么保持思考、不紧张。

建议：

1. 问题不是有没有项目，而是能不能把项目讲深

- 面试里，浅项目也能打；
- 讲不深，再新也没用。

2. 面试准备的核心，是“预判问题”

- 不要等面试官现场问；
- 要自己先从项目出发，把可能问题推一遍。

3. 所有回答都要结构化

- 场景是什么；
- 为什么这是个问题；
- 你怎么设计；
- 为什么这么设计；
- tradeoff 是什么。

4. 补内容要贴项目补，不要为了补而补

- 缺后端，就补当前项目能延展出来的高并发、高可用、缓存、削峰、降级这些；
- 缺 agent，就补工作流、记忆、工具调用、产品设计、评测这些；
- 一切围绕“简历里这段经历怎么被追问”展开。

记录

博主你好，我想问下没有实习可以找到暑期吗？然后目前简历上是俩个我感觉比较基础比较过时的项目，但是最近又有很多面试，我想问我需不需要再添加一个技术点新一点的项目，还是先吃透好好准备已有项目？

昨天 23:50

"Yu"撤回了一条消息

主要感觉时间很紧，都在这几天连着，然后今天面了百度也是第一次面试感觉发挥很差，后天又是蚂蚁，大后天阿里，我想问问怎么可以快速提高面试的能力，以及在面试中保持思考呢，感觉还是面少了有点紧张

可以呀 暑期就是没实习也能找



简历发我看下



Yu: 博主你好，我想问下没有实习可以找到暑期吗？然后目前简历上是俩个我感觉比较基础比...

教育经历

北京邮电大学 · 电子科学与技术 · 硕士
北京邮电大学 · 应用物理学 · 本科

2024.09 - 2027.06
2019.09 - 2023.06

项目经历

面向运维巡检的 AI Agent 可编排平台 2025.10 - 2025.11

项目描述: 负责设计并实现一套配置化 AI Agent 可编排平台，将模型、Prompt、RAG、MCP 工具和调度能力解耦建模，支持多策略执行、运行时动态装配及知识增强，提升知识检索、监控分析和业务处理效率。

技术栈: Spring AI (RAG、MCP、Advisor)、Spring Boot、MyBatis、MySQL、PostgreSQL + PGVector、

Redis、React、flowgram.ai、Docker

• 基于 DDD 拆分 api / app / domain / infrastructure / trigger / types 多模块架构，完成 Agent 运行过程中的资源抽象与配置化建模。

• 设计并实现 Auto / Flow / Fixed 三类智能体执行模式，支持“分析 → 执行 → 监督 → 总结”闭环，以及“工具分析 → 规划 → 步骤解析 → 顺序执行”的流程编排。

• 设计运行时装配链，基于数据库配置动态构建 OpenAI Api、OpenAI ChatModel、Advisor、McpSyncClient 和 ChatClient。

- 并完成 Spring 容器动态注入与能力预热。
- 实现 ChatMemory 与 RagAnswer 两类 Advisor，完成文件知识库上传、Tika 文档解析、文本切分、向量化入库及标签化检索，增强上下文记忆与知识召回能力。
- 接入 sse / stdio 两类 MCP 服务，统一编排搜索、通知、日志检索、监控分析等外部工具能力，对接 Prometheus 指标查询与 ELK 日志追踪能力，支持告警后自动拉取指标波动区间与关键错误日志进行初步归因分析。
- 实现画布 nodes / edges 到执行配置的解析逻辑，并基于 TaskScheduler 构建动态任务调度框架，支撑 10+ 定时任务配置，任务执行成功率 95%+，人工巡检与问题初步定位耗时缩短约 50%-60%。

企业RAG智能问答助手

2025.12 - 2026.03

项目描述：面向企业文档场景的RAG智能问答系统，构建基于RAG的智能问答系统，支撑5000+文档的实时语义检索，HitRate@10达92%，端到端查询延迟800ms以内。

技术栈：Python、LangChain、ChromaDB、BM25、Cross-Encoder、Ragas、Azure OpenAI

- 搭建文档 Ingestion Pipeline，贯通 PDF 解析、Markdown 转换、语义分块、Metadata 增强、Embedding、向量入库全流程；引入 Vision LLM 生成图片语义描述并并入 Chunk，实现图文统一索引与跨模态检索。
- 主导混合检索链路设计，融合 BM25 + Dense Embedding 双路召回，结合 RRF 融合排序与 Cross-Encoder 重排序优化检索效果，推动 Top-10 命中率提升约 25%。
- 建立评估与回归测试体系，集成 Ragas + 自定义评测指标，构建 Golden Test Set 回归机制，覆盖 Faithfulness / Relevancy / Recall 等核心维度，累计覆盖 1200+ 测试用例，提升系统稳定性与迭代可靠性。
- 基于 Skill 机制驱动研发流程，设计 auto-coder / qa-tester / setup / package 等 Agent Skill，覆盖编码、测试、配置、打包等流程。

专业技能

- 熟悉 Python、Java，具备 AI 应用开发 + 后端工程落地能力。
- 熟悉 LangChain、Spring AI、OpenAI API、MCP Protocol，具备 Agent 编排、Tool Calling、Prompt 工程、外部工具接入等实践经验。
- 熟悉 RAG 系统设计与优化，掌握 文档解析、Chunking、Metadata 增强、Embedding、向量检索、BM25 混合召回、Rerank 等关键技术。
- 熟悉 多 Agent、Workflow 设计与实现，能够完成任务拆解、执行链路编排、运行时动态装配和闭环调度。
- 熟悉 Ragas、Golden Test Set、效果评测与回归测试体系，能够基于数据指标持续优化问答效果与检索质量。
- 熟悉 MySQL、PostgreSQL、PGVector、Redis、ChromaDB，具备关系型数据建模、缓存设计与向量存储应用能力。
- 熟悉 Linux 操作系统基本命令，熟练使用 Git 进行版本控制与协作开发，熟悉 Docker 容器化部署。

那你就是没准备充分



Yu：主要感觉时间很紧，都在这几天连着，然后今天面了百度也是第一次面试感觉发挥很差， ...

这是我的简历

两个项目你最熟悉哪个 被问的最多的是哪个



今天主要问的第一个

第二个比较熟悉吧

昨天 23:55

这个是 xfg 吗



是的是的

第一个是 xfg 的 agent 项目

快速提高的话，你需要提高你提问的能力。具体而言就是，你要对着你的项目描述，一个点一个点的去想，面试官可能会问什么问题，然后自己整理一下。问的问题，你需要整理出：1. 场景（解决掉的问题）2. 方案（我是怎么做的）3. 我的考量（tradeoff，为什么不用其他设计）

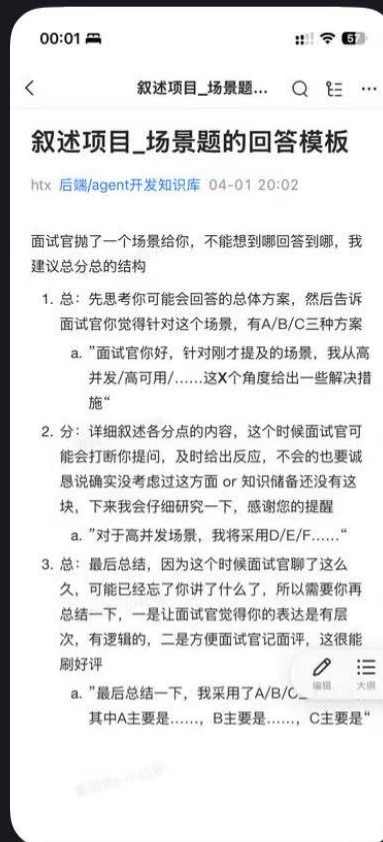


00:00

你要去推演这个过程，人在面对未知点事物是容易紧张的



面试中保持思考：要有固定的答题思路和素材库



答题思路可以参考这种，每个人叙事习惯不同，我比较啰嗦，都是总分总。



素材库：其实就是你知识的广度和深度，之前后端主要就是怎么设计一些高可用高并发的场景。现在 agent 更多的是很多优秀的产品设计你有没有去思考学习过，比如 claude code/openclaw



大厂后端面试，到底怎么准备场景题

场景题是面试里最能拉开差距的部分，八股背得再熟，遇到开放性场景题还是会懵。这篇文章的方法，核心是借鉴北美 System Design 的思路来拓宽视野，虽然国内不考 System Design，但回答场景题帮助很大。

什么是场景题？为什么难？

面试官给你一个实际业务场景，让你设计解决方案。比如：

“你的系统突然流量暴增10倍，你怎么处理？”

“一个秒杀系统，需要考虑哪些问题？”

“反馈接口响应很慢，你会从哪些方面排查？”

为什么场景题难？

标准答案：不像八股文有固定答案，场景题考察的是你的思维方式

全局视角：不能只想到一个点，要从多个维度考虑（高并发、高可用、数据一致性等）

实战经验：光背八股不够，得真正理解技术在实际场景中怎么用

准备的时候也很头疼，看面经发现每个人的回答都不一样，不知道该怎么准备。后来摸索出一套



大厂校招后端面试，到底怎么准备场景题



黄同学 h

后端感觉最近准备的比较薄弱，而 agent 感觉还是准备的比较充分

这个问题不错，我待会可以发群里吗



黄同学h：面试中保持思考：要有固定的答题思路和素材库

可以可以

那你细细品一下我给你发的笔记还有截图的知识库文章



Yu：后端感觉最近准备的比较薄弱，而agent感觉还是准备的比较充分

如果术明白了，接下来就是补充内容的事情了



刚刚

补充内容也不要乱补充，尽量跟你项目贴近，你做秒杀就要补充高并发的场景（类似的举个例子）



简历上的项目怎么选&组合

针对当前的就业环境，我的判断是，AI 已经是在实习简历里越来越应该正面出现的能力。

项目选择上，合理的路线不是只守着传统后端，也不是一一定要做后端转 Agent 这种叙事，而是两种都成立：

要么做后端 + Agent 的组合，用后端项目证明工程基本盘，再用 Agent 项目证明你能把模型接进真实系统；

要么干脆走纯 AI / Agent 路线，但项目不能停在套壳和调用 API，得能证明你在工程问题上做过实打实的东西，而且最好有贴近热点的真实场景，比如 code agent，比如 open claw 等，不要去追求过分复杂业务的 xx 助手。

Q1：目前Java做了黑马点评，如果再学习一个agent项目，简历上一个Java、一个是python，可以找实习吗？会不会有点奇怪

可以的，也并不奇怪。现在这个阶段，语言本身不是核心问题。尤其在 AI coding 往前推得很快的背景下，企业更看重的是你有没有工程能力、有没有做过真实项目、能不能把东西落地，而不是盯着你简历上为什么一个项目是 Java、一个项目是 Python。

点评证明你有后端背景，知道 Spring Boot、MySQL、Redis、缓存、高并发这些东西

Python Agent 项目，证明你不是只会传统后端，对 Agent 也有理解

这是很常见也很顺的一条线，前提是你自己要把这条线讲明白。

你对外的叙事不要变成我会两种语言，而要变成：我有后端工程基础，也在补 AI / Agent 落地能力。

如果你投 Java 后端实习，完全可以带一个 Python Agent 项目，不会减分，反而可能加分。但前提是主线还是 Java，黑马点评最好做一些自己的改造，不要只是培训班原版。

如果你投 AI 应用 / Agent 实习，这个组合也 OK，这时候 Java 项目不是累赘，反而能证明你有工程底子，不只是会调 API。

如果你走纯 Agent 路线也没问题，只是项目不能太 demo。

比语言统一更重要的，是你的项目有没有统一叙事，以及跟岗位的 match 程度。

安全背景的Agent简历

简历

项目经历

基于LangGraph的研究型AI Agent系统

2026年01月 - 2026年03月

- 参考 DeerFlow 的多阶段 Agent 架构，设计并实现面向复杂信息任务的智能工作流系统，围绕任务理解、问题拆解、信息检索、内容筛选、短期记忆管理与结构化结果生成等关键环节完成核心模块设计与整体流程搭建。
- 基于 LangGraph 进行多节点工作流编排，使用状态驱动方式组织任务阶段流转，协调检索、上下文管理与结果生成等模块协同执行，并通过模块解耦降低任务流转逻辑与具体功能模块之间的耦合度。
- 融合检索增强与结果生成流程，实现从开放式任务输入到外部信息获取、上下文组织和结构化分析输出的自动化处理。

基于卷积神经网络的指纹编码技术及应用 | 本科毕设

2024年01月 - 2024年06月

- 实现基于 CNN 指纹特征提取与模糊金库的生物密钥绑定系统，完成指纹灰度图像预处理、有向中心点提取、旋转对齐与裁剪、CNN/孪生网络训练、特征向量生成以及密钥绑定与恢复流程开发，实现基于可再生指纹特征的区块链私钥恢复。
- 基于公开指纹库与自建指纹库完成系统测试与实验对比，并传统基于指纹特征点的编码方案进行比较，从错误接收率、错误拒绝率和时间开销等维度评估性能，验证了系统的可行性与实用性。

其他出于隐私略去

总结

最大的问题：简历内容的岗位指向性不够强。如果按企业视角来看，读完之后会有点拿不准你到底更想投什么：Agent / 隐私计算 / 传统安全方向。这些内容你都沾了，但没有一条明显的主线。内容确实做了很多，能看出有很强的学习能力和自驱力，但到hr眼里可能就是岗位不匹配。

比较明显的问题有这么几个：

- 主线不清楚**：这份简历最前面的项目是 **LangGraph 的研究型 AI Agent 系统**，但后面大篇幅又是非线性函数多方安全计算、纵向联邦逻辑回归、TEE 隐私计算。对大厂面试来说，这种都写一点通常不如有一条明确主线有效。
- Agent 项目描述没到位**：现在的描述像功能说明，建议补上
 - 具体场景是什么
 - 你负责哪部分
 - 跑了什么任务
 - 用什么指标评估
 - 最终效果怎么样
- 几乎没有量化结果**：这是这份简历一个很大的短板。现在大部分经历都在讲做了什么，但很少讲结果怎样。
- 科研内容很扎实，但和岗位关系没讲清楚**。如果你投的是 Agent 或大模型应用岗，这些内容就需要

解释清楚它们和目标岗位的关系。现在的问题是它们占了不少篇幅，但没有帮你把我适合这个岗位讲得更清楚，反而稀释了前面的 Agent 主线，影响hr筛选。

5. 整体表达偏学术，不符合互联网的简历叙事。

可以参考这种表达：

- 做了什么系统
- 解决了什么问题
- 支持什么能力
- 指标提升多少

整体来看整份简历有内容，但没有把内容收成一条足够清楚的求职主线；有技术点，但没有把结果和落地性讲出来。

建议修改的方向：

- 把 Agent 项目突出。
- 安全计算 / 联邦学习 / TEE 相关内容压缩，只保留最能体现技术深度的部分
- 每段都补上结果和指标
- 删除大部分低价值自我评价

大模型算法背景

简历

项目经历

- MedAgent-RAG:面向医疗导诊与检查前准备的多智能体检索增强系统**
- 面向医疗导诊、检查前准备、检验报告初步解读与流程问答场景，设计并实现集成 Qwen2.5-7B-Instruct 微调模型、长短期记忆、RAG 知识库与 MCP 工具调用的多智能体 Agent 系统，提升复杂医疗问答场景下的回答可靠性、任务完成率与安全性。
 - 以单 Agent + 向量检索为 baseline，针对复杂多跳任务中存在的工具路由不稳定、参数填充偏差、长上下文信息丢失与高风险问答拒答不足等问题，基于 LangGraph 构建 Supervisor / Knowledge / Tool / Safety / Response Agent 架构，并引入 Plan-Execute-Replan 工作流与 ReAct 推理机制，形成 badcase 驱动的多轮迭代闭环。
 - 构建医疗垂类知识库，收集 2,300 篇原始医疗文档，清洗后保留 1,950 篇，切分为约 12,200 个 chunks，覆盖导诊规则、检查前准备、常见检验指标说明、医院流程与患者教育材料；同时支持检查单/报告文本解析与结构化抽取，检索侧采用 BM25 + Dense Retrieval + Rerank，并结合 query rewrite、metadata filter、evidence aggregation 与低置信拒答 提升证据覆盖率与一致性。
 - 基于 Qwen2.5-7B-Instruct 完成 LoRA 微调，构建 5,800 条 SFT 样本，覆盖导诊问答、检查前准备、报告解读、工具调用轨迹与安全拒答样本；同时设计短期记忆（最近 8 轮对话 + 会话摘要压缩）与长期记忆（结构化患者画像与既往记录），并通过工具结果缓存与相似查询复用降低重复调用成本、增强多轮追问与个性化问答能力。
 - 基于 MCP 接入 6 个本地/远程工具，支持患者画像查询、报告解析、检查准备查询、预约查询、药物相互作用检查与导诊规则匹配；针对 tool description / schema 重写工具描述、参数定义与 few-shot 调用示例，降低无效调用与参数偏差，并构建覆盖工具评测 + 回答评测 + 人工抽样的评测体系（960 条离线样本 + 120 条人工抽样集），最终相较单 Agent baseline 实现 Correct Path Rate +12pct、Parameter Accuracy +10pct、Task Success Rate +10pct、Pairwise Win Rate 62%，并将 Unsafe Answer Rate 从 7% 降至 3%。

实习经历

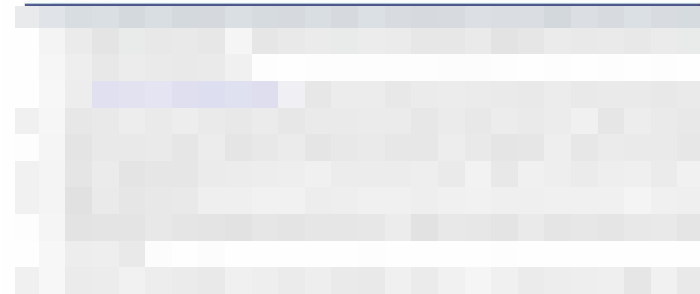
- 2025.09-至今** **清华大学智能产业研究院(AIR)**
- 面向高分辨率 RAW 图像与视频压缩场景，围绕率失真权衡、4K 数据高计算开销与高效编解码需求，探索面向压缩任务的结构建模与推理优化方案。
 - 基于开源压缩框架完成 RAW 数据适配、模型复现与训练流程搭建，引入选择性状态空间上下文建模与能量感知特征增强模块，提升高分辨率场景下的压缩质量与推理效率。
 - 完成多数据集训练、对比实验与消融分析，系统评估不同上下文建模与选择性计算策略在重建质量、码率控制与编解码时延之间的效果权衡。
- 2025.05-2025.09** **中国科学院北京** **算法工程师实习生**
- 面向供应链预测与库存风险预警场景，针对物流时间序列数据噪声强、关键类别误差高、多目标任务难以协同优化的问题，探索 Transformer 在复杂时序任务中的建模与训练优化策略。

- 处理并工程化 50GB+ 物流时间序列数据，完成数据清洗、异常处理、特征构造、样本切片与训练集组织，提升高噪声场景下的数据可用性与训练稳定性。
- 复现 5 个主流时序模型 作为 baseline，围绕 Transformer 类模型持续迭代训练策略、超参数配置与实验流程，并在多任务学习框架下优化损失加权与样本抽样策略。
- 结合 百万级用户行为数据 开展因果分析，辅助定位影响预测与风险预警的关键行为因素，反推特征设计与训练重点。
- 最终关键类别预测 MAPE 降低 15%，关键业务指标较基线提升 10%，库存风险预警准确率提升 8%，验证了 Transformer 与多任务学习策略在高噪声供应链预测场景下的有效性。

- 2025.03-2025.05** **百度,北京** **算法工程师实习生**
- 面向医疗文本理解场景,围绕大规模医学语料在**标注一致性、样本偏差与后处理校验**上的问题,搭建可支撑 NLP / 大模型微调的数据与评测闭环。
 - 基于 **100k+ 中文标注语料** 完成语义表征模型监督微调,将业务问题抽象为 **Query 意图识别与语义匹配任务**,持续迭代训练流程、特征增强与样本组织策略。
 - 分析 **TB 级用户日志**,对异常查询进行偏差归因与 **badcase** 聚类,识别**语义歧义、表达缺失、意图偏移**等问题模式,并将问题回流到数据清洗与训练样本构造阶段。
 - 结合 **100+ 在线异常案例** 与小流量 A/B 测试验证优化收益,形成**“离线微调—线上评测—问题回流”**的迭代闭环。
 - 最终模型 **F1 提升至 0.92**,为下游搜索排序系统贡献 **30%+ 有效新特征**;定位约 **5% 意图识别偏差样本**,使相关异常事件率下降 **20%**,有限流量实验 **CTR 提升 2%**。

- 2024.05-2024.09** **京东,北京** **数据科学实习生**
- **项目背景**:面向仓储与用户运营场景,针对底层数据异常影响分析准确性、用户行为特征表达不足的问题,开展数据治理与用户分层分析,为后续推荐、排序及智能运营类模型提供稳定的数据底座。
 - **技术工作**:编写 **SQL** 脚本开展仓储数据异常检测与修复,定位关键字段质量问题并提升数据一致性;完成 **1M+** 活跃用户的 **RFM** 分层、核心业务指标看板搭建与购买漏斗分析,沉淀用户行为分析与特征抽取流程,增强后续算法建模可用的数据表达能力。
 - **项目结果**:检测并修复 **5%+** 仓库数据异常,提升数据可信度与分析稳定性;形成可复用的用户分层与行为分析结果,为推荐、排序与运营优化类任务提供数据支撑,也补强了你在大模型应用岗位中非常重要的数据理解与数据治理能力。

科研成果



Zhou(Medical Image Analysis Under review 中科院一区)

自我评价

- **编程与算法框架**:精通 Python、SQL, 熟练掌握 PyTorch、TensorFlow 框架, 熟悉 C++、Java, 了解汇编语言; 深入理解 Transformer 架构及大模型相关算法原理, 具备模型微调、数据处理及算法优化能力。
- **校园经历丰富**:参与 4most、野村证券技术洞察日, KCL 新兴领袖项目、科技社, Forage 软件工程及技术开发相关工作模拟并获取相关证书 3 份; 兼任 KCL 科学辅导员、多次参与包头马拉松、大型音乐会志愿活动。
- **语言能力**:母语为普通话, 具备流利的英语书面与口头沟通能力。

总结

hihi uu, 我看完这份简历, 最大的感受是你不是没内容, 反而是内容太多了, 但现在看不太出来你到底想投什么岗位。

如果你想投大模型算法工程师, 这份简历现在的问题是主线有点散。大模型、医疗 NLP、时序预测、图像方向、数据科学你都写进来了, 信息很多, 但 HR 和面试官第一眼其实很难抓住你最强的点, 也不容易判断你和岗位的匹配度。

所以我觉得你最需要先做的一件事, 就是先明确投递方向, 再按方向收束简历。要是你几个方向都想投, 我会更建议你拆成几版简历, 不要全放在一份里。

具体几点我觉得可以改:

- 1. 实习和项目建议换一下位置。工业岗一般还是先看实习, 尤其你实习已经不少了, 实习的说服力会

比项目更强。

2. 项目和科研成果要收一下。现在内容很多，但有点散，建议只保留和目标岗位最相关的项目、论文，别把所有东西都堆上去。
3. 每段经历不要写太满。你现在很多 bullet 技术词很多，但亮点不够聚焦，最好改成“你做了什么、解决了什么问题、最后结果怎么样”这种写法。
4. 如果目标就是大模型方向，那像京东这段、还有一些偏图像/时序的内容，可以弱化一点，不然会显得方向不够集中。
5. 自我评价那部分可以再压缩，留更多空间给实习和项目。
6. UCL 那段如果还没正式入学，建议写清楚已录取或者即将入学，不然时间线看起来会有点怪。

整体上我会觉得你这份简历其实是有内容的，也有亮点，但现在最大的问题不是内容不够，而是没有把亮点收成一条线。先把岗位方向定清楚，再把最相关的经历往前放，效果会好很多。

有算法背景，但主线发散

简历

实习/项目经历

- > 离线云计算资源调度自优化（高校合作项目）

华为云算法创新Lab

2025.08 - 2025.12
- **项目描述：**针对云资源调度需平衡SLA与QoS等约束下资源分配的问题，设计构建一个智能体系系统，实现调度策略自动化优化，目标提升资源发放效率同时保障约束合规。

▪ **项目内容：**设计并实现了基于LLM的云资源调度算法自动优化系统，利用AlphaEvolve框架定制岛屿独立策略演化引擎，创新性地构建了多算子协同演化机制与instructor细粒度反馈系统，通过Qwen、DeepSeek等大模型进行代码变异(mutation)与交叉(crossover)优化，引导LLM持续迭代优化调度算法，实现算法自进化闭环。同时采用探索-利用权衡策略构建prompt，结合多模型协同避免局部最优，成功在满足SLA指标（健康主机违约占比、租户密度违约占比）和QoS指标（主机负载超阈值占比）前提下，50台服务器规模下提升CPU核数发放量3.36%，200台规模下提升2.74%，相关成果形成专利并通过评审。
- > Pico：本地代码智能体Harness

大模型Agent

2025.12 - 2026.03
- **项目描述：**面向代码仓库长链路任务开发本地代码 Agent Harness，覆盖模型接入、工具调用、上下文管理、结构化记忆、运行审计与评测闭环，重点解决多轮任务中的上下文膨胀、重复读取、工具误调用和结果不可复现问题。

▪ **项目内容：**(1) Agent Harness 架构设计：负责本地代码 Agent 的整体设计与开发，统一模型接入、工具执行、会话状态和运行工件落盘流程，形成可回放的任务执行链路；支持 2 类模型后端、7 类工具和 3 类运行工件；(2) 长上下文管理：针对代码仓库长链路任务中的 prompt 膨胀问题，设计分层上下文管理与预算裁剪机制；在 12 组真实长上下文配置上，将平均 prompt 长度从 6964 压缩到 5418，平均压缩率 18.01%，最高压缩率 35.63%；(3) 结构化记忆系统：将任务摘要、文件摘要和会话笔记分层管理，并结合 freshness 校验和相关记忆召回减少重复读取；在 12 个真实记忆依赖任务中，将重复读文件次数从 8 次降到 3 次，平均工具步数从 0.67 降到 0.25，任务正确率从 66.7% 提升到 99%；(4) 评测与审计闭环：建立固定 benchmark 与运行审计体系，支持 pass_rate / attempts / tool_steps / failure category / trace 的自动汇总与回归对比；当前覆盖 6 个标准化任务和 86 条自动化测试。

相关技能

- 熟练使用Python、PyTorch框架，具备扎实的算法实现与工程开发能力，熟悉高性能计算与系统级优化；

▪ 熟悉Transformer架构及Attention机制等大模型核心技术，熟悉主流开源模型（如Qwen、DeepSeek）的结构与微调方法，了解RL基础以及PPO、DPO、GRPO等强化学习算法

▪ 熟悉大模型Agent技术栈，包括Prompt工程、上下文工程、多智能体协同、MCP、RAG、LangChain与算法自动生成（如FunSearch、AlphaEvolve）等前沿技术

▪ 通过英语四六级，CET4-602分，CET6-575分，具备良好的英语读写能力

结论

算法背景其实不错，可以考虑一下投算法岗哈。

你现在约面少的原因是因为，你的简历主线比较发散。

你有A会，有华为的实习，但是如果是大厂的agent部门，是比较难捞你进面的，因为你上面的内容不够聚焦。

如果你想投递Agent岗位，有几点建议：

1. 实习和项目分开，实习单开一个。同时华为那段可以保留，但要稍微包装一下。这段内容成功上看没问题，但它现在写得有点偏算法机制堆叠，容易让人读完觉得你像在做研究型优化，而不是应用开发。如果你投的是大模型应用开发/agent，这段建议更强调基于 LLM 的自动优化系统落地和最后带来了什么收益。

2. 技能部分建议往应用开发栈收，不要写得太像模型算法研究岗。现在写了 PPO、DPO、GRPO 这些，放着没问题，但它们不应该成为重点。如果你是投大模型应用开发，技能区更该强调的是：Python / PyTorch / Transformer / Agent / Prompt Engineering / Context Engineering / MCP / 等等至于 PPO、DPO 这些可以放在后面一句了解/熟悉，不要抢主线。

所以我总体看，你这个简历可以拆成两份，算法和Agent都可以做，但切记不要都写在一起，都写在一起最终就是很难被hr匹配到关键词，需要业务花时间去细看你的简历（往往大家都很忙没那么有空，所以不写的特别针对，是比较难发面的）

bg不错大厂约面少，字节横向，腾讯一面挂

总结

问题：后端深度不够、AI方向又不够突出

核心判断：

1. 简历本身不是大问题，因为能拿到字节、腾讯面试，说明简历至少过线了。
2. 更大的问题是投递策略和岗位匹配，不完全是“简历写得不好”。
3. 他的背景本质上还是“传统后端 + 一点 RAG/AI”，没有特别强的差异点和记忆点。
4. 他后端项目的深度还不够，而现在后端岗位很多是存量需求，要求更严格。
5. 他如果继续硬卷后端，再补一个后端项目意义不大，不如往增量市场，也就是 Agent/AI 应用方向走。

具体意见：

1. 优先复盘字节二面、三面，重点看两个问题：
 - 项目有没有讲出深度
 - AI coding、解决问题能力、场景题表现有没有展示出来
2. 区分两类问题：
 - “复盘字节面试”是解决横向/后续推进的问题
 - “投什么岗位”是解决进面率的问题
3. 投递策略建议：
 - 有明确后端岗位的公司，优先投后端开发/后台开发/服务器开发
 - 没有纯后端岗的，比如部分公司，可以投 AI 应用研发
4. 中期策略上建议：
 - 简历改得更偏 agent
 - 但后端岗位也继续投，不要完全放弃
 - 利用当前窗口期补一点 agent 相关内容

记录

黄哥，可以帮忙看看简历怎么修改吗？

我三月底把所有开暑期实习的大厂中厂都投了，但是只有字节和腾讯约面。字节三面结束横向一周了，腾讯2次一面挂。其他公司根本不约面。我准备把简历上的AI项目换成pico

南京大学本科 软件工程 硕士
武汉大学 计算机科学与技术 本科
2023, 2024年全国大学生计算机系统能力大赛-数据库管理系统设计赛：湖北省第二名、全国三等奖
CET-6: 553分

2025.09-2027.07
2021.09-2025.06

项目经历

星罗金-推荐周边抢购平台 后端开发 2025.09-2025.11

项目简介：本项目是面向明星粉丝群体的限量周边抢购平台，专注于高并发场景下的极致购物体验。核心功能涵盖商品浏览、秒杀抢购、订单支付及全链路订单管理。项目通过缓存、异步解耦、限流方案，实现了高可用、高并发的系统架构，支撑秒杀业务场景的稳定运行。

技术栈：Java, SpringBoot, MySQL, Redis, Lua, Caffeine, Kafka, MybatisPlus

项目亮点：

- 高并发秒杀、防止库存超卖与一人一单机制：使用Redis存储商品、库存、订单信息，使用Redis+Lua脚本原子化判断用户下单资格和预扣库存，使用MySQL乐观锁防止库存超卖，使用Kafka消息队列异步处理库存扣减、订单生成和支付，核心接口响应时间降低约70%，提高秒杀场景的并发性能。
- 多级缓存加速与缓存优化：针对热点商品数据，使用Caffeine本地缓存与Redis搭建二级缓存架构，提高热点数据访问速度，使用逐条过期+独立线程重建缓存方案防止Redis热点key缓存击穿，使用Redisson布谷鸟过虑器方案防止Redis缓存穿透，使用熔断TTL方案防止缓存雪崩，降低数据库压力。
- 滑动窗口限流：使用Redis+AOOP+注解实现支持全局/IP/用户/用户ID多维度的限流机制，有效抵御爬虫和黄牛脚本，防止系统过载，保障真实用户体验。
- 超时未支付订单的定期自动关闭：使用Spring Task定时任务+Redis Zset实现超时未支付订单的快速查询和到期自动关闭，释放未支付订单库存，提升库存周转率。
- 支付和关单的并发：使用乐观锁解决订单支付回调和超时关单状态下的并发问题，确保订单状态流转的准确性和一致性。
- 数据一致性保证：更新数据后删除Redis缓存，若删除失败则使用消息队列重试，使用Redis TTL兜底保证数据一致性。

云栖AI Agent-研发团队知识库智能系统 开发 2025.12-2026.02

项目简介：本项目是基于Spring AI构建的面向研发团队的智能AI Agent系统，聚焦研发场景下的知识管理与任务自动化需求，实现了任务自主决策、研发工具调用和研发知识库精准检索功能。

技术栈：Java, SpringBoot, Spring AI, PostgreSQL, pgvector, Redis, MybatisPlus, React, Typescript

项目亮点：

- Agent核心引擎：设计并实现Think-Execute循环机制，支持多轮工具调用和任务规划，使Agent能够自主拆解复杂研发问题并逐步求解。
- RAG检索功能：基于pgvector实现向量相似度搜索，支持技术文档、设计文档、API规范等研发资产的语义检索，提升知识获取效率。
- 工具调用框架：提供可扩展的工具调用能力，支持研发常用工具（如文件系统操作、发送邮件、数据库查询、版本控制）的灵活配置与动态启用。
- 多模型支持：设计ChatClientRegistry注册表模式，实现多模型动态切换，可根据任务类型选择最合适的大语言模型。
- 实时交互体验：基于SSE服务端推送，实时更新Agent执行状态、思考过程及中间结果。
- API设计：设计RESTful API和统一响应格式，提供完整的后端服务。

专业技能

- 熟悉使用Claude Code辅助编程提升开发效率，了解提示词和Skills的设计。
- 熟悉Java基础、集合等相关知识，了解常见的设计模式，了解JVM的垃圾回收机制、类加载机制及Java的内存区域。
- 熟悉Java开发常用、了解J2C中常用的工具类，如ConcurrentHashMap等，了解多线程、线程池、Java内存模型。

已阅

码住

好家伙

23:16

点评写的挺好的，应该是我目前看到的最好的点评。你能腾讯字节约面，简历上应该没问题，更多是投递策略的问题，你投其他公司大概都投什么岗位呢

21

就是我总体看，你是在传统后端基础上加了点rag，其实也没什么问题按照你的bg，主要是没什么差异点或者记忆点



你要着重复盘一下你的字节三面二面的内容



复盘的角度是两个地方：1.从项目看 面试官的反馈 你感觉你有讲够深度 2.软素质方面：ai coding/解决问题的能力 这些你有展示过吗 可以是场景题 也可以是你在讲项目的时候的延伸



23:25

hello



谢谢宝



呃



在干嘛



喜欢

发消息...



23:25

这个是解决你横向的问题



复盘的角度是两个地方：1.从项目看

黄同学h: 复盘的角度是两个地方: 1.从项目看面试官的反馈 你感觉你有讲够深度 2.软素质...

这个是解决你进面的问题



黄同学h: 就是我总体看, 你是在传统后端基础上加了点rag, 其实也没什么问题按照你的bg...

你有投固定 base 吗



还是全国可飞



23:34

有后端开发/后台开发/服务器开发的优先选这个, 蚂蚁阿里一类没有后端开发的选 AI 应用研发

黄同学h: 点评写的挺好的, 应该是我目前看到的最好的点评。你能腾讯字节约面, 简历上应...

全国可飞, 没有固定 base

hello



谢谢宝



呢



在干嘛



喜欢

发消息...



理解了



其实就是



你现在后端项目上也不是很深入



还是个点评 虽然写的不错了



现在后端大部分公司是存量 组的要求会稍微严格



阿里也不约面吗



没有约

黄同学h: 阿里也不约面吗

那我接下来是加上 pico, 投 ai agent 相关的岗位吗?

那就是不对口 你的经历其实比较难让业务部门 match 到 ai 开发



23:39

策略是 后端再做个项目我觉得意义也不大了 复杂度摆在那 不如往增量市场走



把简历改的偏 agent



但是后端也投



然后字节多面 能到三面肯定还有其他业务部门捞你



这个窗口期再去补充点 agent 相关的内容



可以的



momo: 那我接下来是加上 pico, 投 ai agent 相关的岗位吗?

个人面经

暑期

阿里暑期一面面经

阿里暑期一面面经

时间：2025.4 电话面

项目

- 你介绍的项目里用了DDD，简单讲下项目里面的领域是怎么划分
 - 按策略，活动，积分，奖品四个领域划分
 - 拿策略具体讲了下细节，策略领域只负责接收策略ID，通过责任链模式（处理黑名单、权重）和规则树（处理库存锁、兜底），计算出最终的“奖品ID”
 - 追问了为什么要这么划分
 - 讲了下痛点，主要是规则的多变性和业务的独立性，基于这个痛点，如果是MVC，不加以控制很容易所有的业务逻辑（比如判断黑名单、计算库存、随机算法、发奖）都堆积在Service层，导致service层臃肿
 - 比如，抽奖策略可能包含黑名单、权重、规则树等多种玩法，如果它和活动库存管理耦合在一起，修改一个概率计算逻辑可能会误触到库存扣减的代码。我们需要一种架构，能让核心的计算逻辑与事务逻辑分离，防止业务逻辑随着需求迭代而腐化。
- 抽奖后的库存扣减是怎么做的
 - 库存预扣减：活动开始前，将库存预热到Redis中，当抽奖请求到达规则树的库存节点时，直接使用 Redis 的 decr 命令进行原子扣减。如果decr后的返回值小于0，说明库存不足，直接拦截请求，走兜底逻辑（如返回积分），并不再穿透到数据库
 - 滑块锁：用 decr 扣减后的返回值作为一个唯一的库存索引，拼成一个key，确保颗粒度最细的原子性
 - 异步延迟队列同步到库表：老生常谈的方案，不赘述了

八股

- 数据库的ACID
- MVCC的实现原理

- 主要讲了下readview里面比较事务id的逻辑
- **redo log是怎么保证Crash Safe 的 **
 - WAL 机制，先写日志再刷盘
- B+树为什么比B树更适合索引
 - 先对比讲了下几种数据结构做索引的缺点
 - B+树非叶子节点不存数据，层高更低，范围查询更优
- redis里zset用过吗，讲一下
 - 讲了zset的底层原理，底层由dict和skiplist组合而成，dict用于建立成员到分值的映射，实现 $O(1)$ 时间复杂度的成员查找与分值获取。而skiplist则通过“随机层高”建立多级索引，实现了平均 $O(\log N)$ 复杂度的快速范围查询与插入，还通过双向指针支持了逆序遍历
- redis持久化讲一下
 - 追问了AOF重写是阻塞的吗（主进程 fork 子进程，不阻塞）
- TCP和UDP的区别，各自适用于什么
 - TCP 是面向连接、可靠的字节流传输，支持拥塞控制和流量控制，适用于文件传输、邮件等。
 - UDP 是无连接、不可靠的数据报传输，头部开销小，实时性强，适用于视频会议、直播和游戏。
- TCP拥塞控制原理
 - 慢启动（指数增长）、拥塞避免（加性增长）、快重传（收到3个冗余ACK立即重传）和快恢复。
 - 目的在于根据网络带宽状况动态调整发送速率，防止网络崩溃
- Java双亲委派模型及打破方式
 - 加载请求先委派给父类加载器，父类无法加载时子类才尝试加载
 - 打破：重写loadClass方法，或者利用ServiceLoader由父加载器调用子加载器
- JVM垃圾回收机制与收集器的区别
 - 一大波八股吟唱，收集器主要讲了CMS和G1的区别

更新: 2026-01-28 11:39:57

原文: <https://www.yuque.com/u63312805/wnyutc/yfaelpptnm09yimn>

美团暑期二面面经

美团暑期二面面经

项目

1. 介绍一下你项目的一个核心场景是怎么实现的

- a. 我选的简历上的点评秒杀场景
- b. 背景：针对热点优惠券的秒杀活动，面临高并发读写压力
- c. 流程：
 - i. 用户资格校验：请求到达后，利用Redis的BitMap判断用户是否重复下单，实现一人一单。
 - ii. 库存预扣减：使用Lua脚本原子性地执行get库存、判断大于0、decr扣减库存，解决超卖问题。
 - iii. 异步下单：扣减成功后，将订单信息（用户ID、优惠券ID）发送到消息队列
 - iv. 落库：后台线程从队列中获取消息，异步写入MySQL数据库（创建订单、扣减数据库库存）。

2. 项目有没有做压测，怎么做的

- a. 用 JMeter做了压测，主要是对比秒杀。
- b. 原始方案（直接查数据库+synchronized 锁）在 1000 并发下 QPS 极低且容易超时。
- c. 优化后（Redis 缓存+Lua 脚本+异步队列），QPS 提升到了 2000+），且响应时间显著降低

3. 还有什么办法可以提高秒杀的并发量

- a. 临场想了个热key分片
- b. 单个Redis节点处理热点Key（如极热门的优惠券）可能有瓶颈
- c. 将热key分成若干子key，请求到来时，随机路由到一个子Key上进行扣减

八股

1. MVCC原理

- a. 隐式字段：InnoDB 给每行数据默认加上了最近修改（插入/更新）事务 ID、回滚指针、隐式

主键 三个隐藏字段

- b. 当事务修改数据时，MySQL不会直接覆盖旧数据，而是把旧数据记录在Undo Log中，通过回滚指针，将新老数据串联起来，形成一个**版本链**
- c. 同时read view 可以用于判断版本链中哪个版本对当前事务可见，比对版本事务id就完事了

2. Mysql日志

- a. Binlog、Redo Log、Undo Log

3. kafka怎么保证消息不丢

- a. 生产者端： 设置 acks=all （或 -1）
- b. Broker 端： 设置多副本和至少写入 2 个副本，禁止选举非 ISR 节点为 Leader。
- c. 消费者端： 关闭自动提交，业务逻辑处理成功后，手动提交Offset

4. push模式和pull模式的优劣，为什么kafka是pull

- a. 优点： 消费者可以根据自己的处理能力主动拉取数据，避免被由于生产速度过快被压垮，提高吞吐量
- b. 缺点： 如果队列为空，消费者可能会陷入死循环空轮询
- c. kafka 通过fetch.min.bytes 和长轮询阻塞机制优化了这个问题)

5. hashmap底层原理

- a. 吟唱八股，直接截了自己整理的八股笔记内容

- b.  图片加载失败

AI相关

我投的不是后端吗，面试官突然问我有没有了解AI，最近有没有用AI做什么

给他讲了RAG的一个小项目和自己用AI辅助编程

后续自己又补充了点相关的内容，主要是agent相关

手撕

leetcode92 反转链表2

 图片加载失败

更新: 2026-01-28 11:39:42

原文: <https://www.yuque.com/u63312805/wnyutc/zofzngrhulurtcf5>

美团暑期一面面经

美团暑期一面面经

时间：2025.3 部门/业务：核心本地商业-业务研发平台

项目

1. 点评项目里面redis用来做什么

- a. 缓存：缓存商铺类型、商铺详情，降低 MySQL 压力，提高查询 QPS
- b. 分布式锁：优惠券秒杀，setnx
- c. 消息队列：秒杀异步下单，用户点击秒杀->校验资格->发送消息到 Stream->立即返回排队中状态>后台线程组从 Stream 读取消息->异步操作数据库创建订单

2. 这个过程中有没有遇到mysql和redis的一致性问题，怎么解决的

- a. 首先讲了几种常见的缓存一致性方案，并简单分析了一下优劣
- b. 讲了项目里用了旁路缓存，在更新商铺信息时，选择先写 MySQL，成功后再删除 Redis 中的 Key

3. 项目里的消息队列有什么用

- a. 解耦：生产者和消费者不需要直接依赖，比如秒杀中，主业务只负责判断资格，下单业务完全独立
- b. 异步：耗时操作移出主流程，降低响应时间
- c. 削峰填谷：在流量高峰（如秒杀开始瞬间），请求积压在队列中，消费者按照自己的能力慢慢处理，防止数据库被打挂

八股

1. **synchronized是什么？能修饰什么 **

- 一个java关键字

2. volatile有什么用？

- a. 保证可见性：一个线程修改了volatile变量，其他线程能立即读取到最新值
- b. 禁止指令重排序：但不保证原子性

3. ** ThreadLocal听过吗，原理知道吗**

- a. ThreadLocal提供了线程局部变量，每个线程拥有自己独立的副本，互不干扰
- b. ThreadLocal内部通过ThreadLocalMap存储数据。ThreadLocalMap是Thread类的一个成员变量
- c. 这里还讲了点在点评里面做用户登录拦截的时候用了ThreadLocal
- d. 最后还问了下结束需要干什么，要调用remove()防止内存泄漏

4. 线程池有哪些参数，有什么用

- a. corePoolSize、maximumPoolSize、keepAliveTime、unit、workQueue、threadFactory、handler八股吟唱

5. 类加载过程

- a. 加载、链接、初始化，依旧八股吟唱

6. java三大特性

- a. 继承、封装、多态

7. 给了个sql，大概就是考察最左匹配

- a. 在使用联合索引（如（a, b, c））时，查询条件必须从索引的最左边列开始匹配，且不能跳过中间的列。如果遇到范围查询（>, <），则右边的列索引失效
- b. 之后又问这个sql会不会回表，然后怎么优化
- c. 这里结束就开始手撕写sql了

手撕

1. leetcode 199二叉树右视图



图片加载失败

2. 比较简单的mysql，没有记录

更新: 2026-01-28 11:39:33

原文: <https://www.yuque.com/u63312805/wnyutc/bsh6zfw894rgwuy>

字节暑期二面面经

字节暑期二面面经

时间：2025.3

项目

- 介绍ddd架构，为什么要用这个，有什么考量
 - 主要讲了充血模型、贫血模型，和MVC的对比、
 - 考量的话从新业务引入和后端视角看法跟面试官分享，着重讲了ddd中的**依赖倒置**
 - MVC 的贫血模型结构设计，数据库持久化对象，很容易被当做业务对象使用，这样后期难维护
 - DDD 的分层结构是以 domain 领域实现为核心，一个 domain 领域下所需的外部服务，都由领域层定义接口，让基础层做具体实现。
- 想要设定每天内只抽出一个大奖，怎样实现
 - 把这个需求实现为规则树中的一个逻辑节点，需要考虑高并发下查询数据库产生的压力和竞态条件
 - 用了Redis 动态 Key + Lua脚本的方案。Key 由奖品 ID 和日期后缀组成，利用INCR的原子返回值判断是否为首位中奖者。如果返回值为1则通过，否则触发规则拦截，让流程流转到底奖励节点。
 - 如果中大奖但是写库失败怎么办？怎么继续保证今天有人中大奖？（玩我是吧！）
 - 核心是做好幂等，如果写库失败了，可以通过异步补偿（比如给这个失败的用户补发奖品）来解决。用一张task表，先写一条状态为“处理中”的任务记录到数据库 如果后续流程全失败了，由**定时任务**去扫描这张表，发现半小时了还是“处理中”，说明出异常了，由定时任务去 Redis 里执行 **DECR** 并把任务标为“已失效”。
- 抽奖的公平性，怎么保证每个人抽到奖品的概率是一致的
 - 面试官纠结的点是，如果有人已经把大奖抽完了，那对后面的人是不是不公平了
 - 因为业务设计上就是**不放回抽样**（抽一个少一个），在这种情况下**先验概率是公平的**，面试官的视角是后验的，这块地方在工程上可以优化的点后面我想了一下，如果奖品个数是多个，可以分时段投放，不在一个时段一次性投放，减少参与时间点不同而导致的后验概率不同

场景题

100万并发量同时抢一亿个红包，怎样设计这个系统

- 因为简历上写了秒杀，面试官让我换个场景设计一下，100万是qps，一亿是红包数，下面是我在面试的时候回答的一些点，我思考的方向主要是，**能不能参考自己简历上的项目，让面试官觉得我做了项目的同时有自己的思考，可以举一反三。**
- 100万QPS肯定不能一次性全打到后端，先加一层限流，过滤一些脚本行为
- 对于正常的流量，可以沿用我大营销项目的思路，**用DDD+规则树处理**
- 一亿个红包的存储，存redis肯定不行，用分桶去存，**将1亿个红包分成1000个桶**，用户请求进来，用`userid%1000`路由到对应的桶，可以有效解决热key问题
- 红包金额的分配，也肯定不能现领现算，会把后端打爆，我的思路是**先预热**，在活动开始前，**将红包金额预先生成好**，存入redis的List中，抢红包操作就简化成了一个LPOP操作
- 抢到之后写库，我的方案是redis抢到后，发送一条mq消息，在内存缓冲区聚合，比如**每500ms or 凑齐10000条数据，就拼成一条sql批量地写**
- 最终一致性，可以利用**task表的补偿方案**，如果mq丢了或写入失败，通过reids里的已发奖总数与DB 记录进行对账，保证不少发

八股

- 对IOC、DI和AOP的理解，它们之间有什么区别
 - IOC是设计思想，原本程序手动创建对象，现在交给Spring容器控制
 - DI是IOC的实现方式，容器在运行期间，将对象依赖的属性通过反射注入进去
 - AOP：与业务无关却被业务共同调用的逻辑封装成切面，减少系统重复代码
- Spring AOP的俩实现
 - JDK动态代理、CGLIB字节码
 - 如果目标对象实现了接口，默认用JDK；否则用CGLIB。也可以配置强制使用CGLIB
- AOP能不能对final类使用
 - 如果是JDK动态代理：可以对实现了接口的final类进行代理，但不能代理类中非接口定义的方法
 - CGLIB就完全不能，因为他通过继承的
- Spring 事务失效的场景
 - 内部方法调用，本质原因是代理对象，可以用拆分service，在service里注入自己，AopContext获取当前代理对象的方法

- 其他的场景比如方法不是public
- Object的方法都介绍一下
 - hashCode(), clone(), wait() / notify() / notifyAll(), getClass(), equals()
- hashCode和equals区别
 - equals 比较的是逻辑上的值是否相等；hashCode 返回一个整数，用于在哈希表中确定位置
 - 如果两个对象equals相等，它们的hashCode必须相同。
 - 如果两个对象hashCode不同，它们一定不等于。
 - 如果 hashCode 相同，对象不一定 equals（即哈希冲突）。
- hash冲突以及解决的方式
 - 链地址法、开放寻址法、再哈希法

手撕

k个一组反转链表，leetcode25



图片加载失败

更新: 2026-01-28 11:39:13

原文: <https://www.yuque.com/u63312805/wnyutc/hb16a49y95sm1qr2>

字节暑期三面面经

字节暑期三面面经

项目

- 先简单讲了整个项目（大营销），着重问了需求的场景和技术选型
 - O(1) 算法具体是怎么实现的
 - 用 SETNX 加锁兜底的话，在高并发秒杀场景下，加锁会让性能退化成串行，怎么保证 QPS
 - **抽奖系统为什么需要 ES **
 - 延迟队列更新 DB，那如果用户秒杀成功了，立刻去查订单列表，发现没有订单，用户投诉怎么办
- 因为简历上有写一些指标性的东西（QPS），就问了很多压测相关
 - 怎么做压测的、压测的结果是什么、瓶颈是什么、怎么改进、改进后的效果
 - 怎么发现是docker带来的性能损耗
 - 为什么docker部署会影响 MySQL 性能？
- 因为这部分问的比较多，答案整理的比较多，我放在了后面

八股 & 场景题

1. 有没有听过分布式ID
2. 讲一下雪花算法，是怎么生成ID的
3. 抖音的评论系统，我们要给每条评论都生成一条id，应该怎么设计
 - 主要解决几个问题，高性能，时钟回拨、id猜测
 - 改良版的雪花算法。通过位运算重组结构来隐藏业务量，通过休眠或扩展位机制解决时钟回拨，必要时引入RingBuffer缓冲层来应对极端流量

开放性问题 & 智力题

1. 有了解 LLM 吗，读研有没有看过相关的paper?
2. 平时会不会看源码，最近有在学什么技术吗？分享一下
3. 智力题，涉及隐私可能开盒就不列出来了

手撕

合并区间的变体，比较像leetcode 435

先用dp做出来了，分析了时间复杂度，提示了下可以用贪心

更新: 2026-01-28 11:39:24

原文: <https://www.yuque.com/u63312805/wnyutc/xq0lfpi4ivmtmdk9>

字节暑期一面面经

字节暑期一面面经

时间：2025.3

项目

- 介绍自己项目的一个亮点
 - 刚好准备了很多大营销项目的话术，讲了用责任链
 - 责任链是在抽奖前执行的相当于做了一层过滤，黑名单用户或者优质用户将会使用不同的抽奖策略进行抽奖。责任链是通过**工厂模式构建**的，责任链会顺序的将责任节点，通过责任链工厂，从库中读取的到的责任节点进行顺序填充到责任链上，写入缓存，下次直接从缓存获取链条。
- 用到了mq，为什么要用mq解耦，然后追问了mq的常见八股，怎么防止消息丢失
 - **业务流程解耦**：比如异步写入中奖记录和发送奖品消息
 - **系统模块解耦**：奖品发放可能涉及多个不同的系统或服务（如RPC/HTTP接口），通过MQ可以将这些系统解耦。发送方只需要将消息发送到MQ，而接收方可以独立地消费和处理消息
 - **防止消息丢失**：结合中奖记录在写一条发送MQ的任务记录，任务记录上有一个状态，标记是否发送完成，这样就可以通过任务扫描的方式完成 MQ 的补偿发送
- 加新的需求，如果想要自动适配多种抽象方式，应该新增什么样的表，加哪些字段

八股

- redis一般什么时候会有较大延迟？
 - **慢查询、读写大key**、生成RDB/AOF重写的时候**fork()**因为内存过大导致的**阻塞、网络**
- 较大的key会导致延迟，那么怎么解决？
 - **查**：redis-cli --bigkeys 找大key
 - **拆**：将一个大的 **hash** 拆分成多个小的 **hash**，或者根据业务 ID 进行分片。
 - **异步删**：不用DEL，用UNLINK异步删避免阻塞
 - **优化**：对于大key可以加一层本地缓存

- redis过期key删除策略是怎么样的？
 - 定期删除+惰性删除
 - 每隔一段时间（默认100ms）随机抽取一部分设置了过期时间的 Key，检查并删除。当客户端访问某个Key时，Redis检查其是否过期，过期则删除。
 - 如果内存还是满的话就触发内存淘汰策略，LRU/LFU八股吟唱
- redis为什么快、简单讲一下跳表是什么
 - 内存操作、数据结构高效（跳表、哈希表）、单线程模型、IO多路复用
- hashmap结构以及crud
 - JDK1.8后是数组 + 链表 + 红黑树
 - 写：计算key的hash值并取模定位数组下标、如果位置为空，直接插入，如果有冲突，先遍历链表或红黑树，key相同则覆盖，不同则插入尾部
 - 读：定位下标后，通过equals方法找到对应的key
- hashmap扩容机制
 - 八股吟唱
 - 创建一个容量为原数组2倍的新数组。
 - **Rehash**：遍历旧数组，将元素搬迁到新数组。
 - JDK1.8 优化：元素要么留在原位置，要么移动到“原位置 + 旧数组长度”的位置，无需重新计算hash。
- 为什么hashmap中要用红黑树
 - 当哈希冲突严重时，链表会变得很长，查询复杂度从 $O(1)$ 退化为 $O(n)$
 - 红黑树的查询复杂度是 $O(\log n)$ 。

手撕

- 给你一个整数数组 `nums`，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和，并输出该子数组在原数组中的起始下标和结束下标。
- leetcode53最大子数组和改版，要求输出起始位置和结束位置，并讲时间复杂度

更新: 2026-01-28 11:39:05

原文: <https://www.yuque.com/u63312805/wnyutc/sqhz8r75rsk3klkg>

个人整理项目笔记

点评

点评

[https://www.yuque.com/g/u63312805/cilpp3/collaborator/join?](https://www.yuque.com/g/u63312805/cilpp3/collaborator/join?token=oQrN7JIJrgP4MdJR&source=book_collaborator#)

[token=oQrN7JIJrgP4MdJR&source=book_collaborator#](https://www.yuque.com/g/u63312805/cilpp3/collaborator/join?token=oQrN7JIJrgP4MdJR&source=book_collaborator#) 《点评整理笔记》

大营销

分库分表

分库分表

分库分表我的理解是项目里拓宽后端技术栈的一个表现，因为对于大营销来说，其实前期没必要分库分表，数据量就那么多，所以分库分表的设计是

因此需要注意以下几个问题

- 为什么分？
- 怎么分？
- 副作用怎么解？

话术

** 你的项目目前数据量有多大？为什么要分库分表？ **

坦白说，项目目前的实际数据量确实还没达到 MySQL 的单表瓶颈（通常是 500w~1000w）。但我进行分库分表的设计，主要是基于对**业务增长的预判和架构演进的考量**

业务特性分析： 大营销系统中的 用户抽奖订单表（UserRaffleOrder）在大促活动期间，QPS极高，且数据量是随时间线性暴增的。

消除架构瓶颈：

1. **I/O 瓶颈：** 如果单表数据破亿，B+ 树高度会变成 3 层甚至 4 层，磁盘 I/O 次数增加，查询性能会急剧下降。
2. **连接数瓶颈：** 单个 MySQL 实例的连接数是有限的，高并发下数据库连接池很容易被占满。
3. **DDL 风险：** 对亿级大表进行加字段或索引操作，会锁表很久，这是生产环境无法接受的。所以，我选择在架构设计初期就引入分库分表，是为了让系统具备承载亿级流量的水平扩展能力（Scale-out）。

追问

**** 你是怎么分的？用的什么组件 ****

为了更深入理解分库分表的底层原理，我没有直接使用 ShardingSphere 等成品中间件，而是基于 Spring AOP + MyBatis 插件自研了一套轻量级路由组件。

- 分片策略：

- 路由键：我选择了 `userId` 作为分片键。因为 C 端用户的查询 99% 都是查‘我的订单’，这样可以确保同一个用户的数据都落在同一个库表中，避免跨库查询。
- 拓扑结构：采用了 2 库 4 表 的设计（共 8 个分片），理论上可以支撑千万级到亿级的数据量。

- 路由算法（核心亮点）：

- 我参考了 HashMap 的源码，使用了 Hash 取模 + 扰动函数的算法。
- 具体公式是：`(size - 1) & (userId.hashCode() ^ (userId.hashCode() >>> 16))`。
- 为什么要扰动？很多用户的 ID 生成是有规律的（比如前几位相同），直接取模容易导致数据倾斜（某些表数据特别多）。通过异或高 16 位，可以让 Hash 值更加散列，保证数据均匀分布在 8 张表中。”

分库分表一定全是优点吗，有没有什么问题，你是怎么解决的

会存在三个问题

一：分表后，原来的自增主键会失效，ID 重复？

- 解决方案：我引入了雪花算法。
- 收益：生成了全局唯一的 64 位 Long 型 ID。不仅保证了唯一性，而且因为 ID 是趋势递增的，插入数据库时能减少 B+ 树的页分裂，保证了写入性能。

二：只能按 `userId` 查，那运营后台要‘查询所有订单’或者‘按时间范围查询’怎么办？

- 解决方案：采用了 Canal + ElasticSearch 的方案。
- 实现：Canal 伪装成 MySQL Slave 监听 Binlog，将分散在 8 张表的数据实时聚合、同步到 ElasticSearch 中。
- 收益：C 端用户走 MySQL 分片（高性能写），B 端运营后台走 ES 索引（复杂读），实现了读写分离和场景分离。

三：跨库后的事务问题怎么解决？

- 解决方案：我在设计时严格遵循了 DDD 的聚合根原则。
- 实现：因为路由键是 `userId`，同一个用户的‘扣减额度’和‘写入订单’操作，肯定会被路由到同一

个库中。因此，核心流程依然可以使用本地事务（@Transactional），避免了复杂的分布式事务（

更新: 2026-01-28 11:37:11

原文: <https://www.yuque.com/u63312805/wnyutc/thtgkvpqd4ftmkwg>

简历撰写参考

简历撰写参考



图片加载失败

多样性C端积分抽奖玩法平台

核心技术： Spring Boot、MyBatis、MySQL、Redis、React、TypeScript、Dubbo、Nacos、RabbitMQ、ElasticSearch

项目描述： 在互联网 C 端业务场景下，为实现高效拉新与用户活跃度提升，打造此抽奖系统。借鉴掘金社区签到抽奖及拼多多大转盘功能，旨在支持涵盖通用抽奖、黑名单过滤、十连抽、积分消耗抽奖、N 次抽奖解锁奖品等丰富且复杂的抽奖场景，满足多样化的营销需求。

核心职责与贡献：

1. 核心架构搭建：基于 DDD（领域驱动设计）思想进行开发，运用模板模式精准定义抽奖流程框架，确保流程的规范性与可扩展性；借助工厂模式高效生成责任链与规则树，使系统能灵活应对不同抽奖规则，实现高内聚低耦合的系统设计。
2. 抽奖算法设计：基于空间换时间思想，实现概率分段映射算法，将时间复杂度从 $O(n)$ 降至 $O(1)$ ，单次抽奖接口响应 $\leq 50ms$ 。借助分治和异步处理的思想，在单抽基础上扩展实现了10连抽的玩法，提高了活动可玩性。
3. 抽奖流程管理：抽奖流程分为前中后三个阶段，抽奖前使用责任链模式进行黑名单、权重规则过滤，抽奖中、后使用组合模式中的规则树模型实现次数解锁、库存扣减以及兜底奖品规则。
4. 库存扣减采用了Redis decr分段消费和SETNX加锁兜底的设计防止库存超卖；对于成功消费的库存，通过延迟队列+定时任务来缓慢更新库存，减轻数据库压力，保证了数据的最终一致性。
5. 针对签到返利、积分变更和奖品发放等流程，利用RabbitMQ将流程解耦；同时为确保消息被成功消费，将消息写入Task任务表，定时扫描将发送失败的消息重新推送到MQ。
6. 采用分布式架构，Dubbo作为RPC框架，ZooKeeper负责服务中心注册，多机实例任务由XXL-JOB进行调度。分库分表数据通过Canal解析MySQL的binlog日志并同步到ElasticSearch进行聚合查询，提高数据查询效率。

更新: 2026-01-28 11:36:22

原文: <https://www.yuque.com/u63312805/wnyutc/uw6chvre5e3pmq2a>

具体业务

具体业务

这一板块是区别于以技术划分的板块，针对简历上点做具体的话术

主要的场景是面试官对着你的简历职责与贡献一个个问



图片加载失败

简历上写了6点，我总结归纳了以下的话术

话术

多变的营销规则（对应第1、3点）

****Q：** 我看你写了基于 DDD 和规则树实现了复杂抽奖场景，能具体讲讲是什么业务场景吗？为什么要这么设计 ******

业务背景： 我们的营销活动玩法非常多变。运营经常提出各种组合需求，比如：“只有积分大于1000且未在黑名单的用户才能抽奖”、“抽奖前3次必不中大奖，第4次才能解锁大奖库存”。如果用传统的 MVC 架构，所有的逻辑（黑名单、库存、次数、兜底）都会堆在一个 `Service` 方法里，写满 `if-else`。每次运营改规则，我都要改核心代码，测试也很痛苦。

我的方案： 为了解决规则多变性，我基于 DDD 将抽奖拆解为三个阶段，并运用了设计模式：

- 1. 抽奖前（资格过滤）：** 我使用了**责任链模式**。将“黑名单校验”、“权重规则校验”封装成独立的 Chain 节点。如果运营要加一个“白名单”，我只需要挂载一个新的节点，不用动核心流程。
- 2. 抽奖后（结果决策）：** 我使用了**组合模式构建规则树**。拿到奖品 ID 后，并不代表真的发奖，还要经过“库存扣减节点”和“次数锁节点”的决策。这就像一棵树，库存不够走左边（兜底），库存够走右边（发奖）。
- 3. 流程骨架：** 我用**模板方法模式**定义了标准的 `performRaffle` 流程，确保所有活动都遵循“前置校验 -> 执行抽奖 -> 后置决策”的规范。

业务成果： 这套架构上线后，新增一种营销规则（比如加一个特定人群过滤）完全不影响其他现有活动，开发效率大幅提高。

高性能抽奖算法（对应第2点）

****Q：** 这个概率分段映射算法，和十连抽是怎么实现的？为什么说时间复杂度是 $O(1)$ ？ ******

业务背景： 在 C 端抽奖中，性能体验是第一位的。传统的抽奖算法是遍历奖品列表，累加概率进行对比，复杂度是 $O(N)$ 。当奖品配置很多，或者支持十连抽这种高频调用时，CPU 计算压力很大，很容易导致接口超时。

我的方案（空间换时间）：

1. **$O(1)$ 查找表：** 我利用 Redis（或本地内存）预生成了一个散列表（Map）。
 - 比如 1% 的概率，我就在 100 个位置中占 1 个坑。
 - 抽奖时，不需要计算，只需要生成一个随机数 `Key`，直接从 Map 里 `Get` 对应的奖品 ID。无论配置多少个奖品，耗时永远是固定的。
2. **十连抽实现：** 基于这个 $O(1)$ 算法，十连抽本质上就是循环调用 10 次原子抽奖。但为了性能，我采用了分治 + 异步的思想，在策略层并行计算（或批量获取），将总耗时控制在极低水平。

业务成果： 经过压测，单次抽奖接口的响应时间稳定在 50ms 以内，即使是十连抽也能做到秒级响应，用户体验非常丝滑。

防止超卖与库存一致性（对应第4点）

****Q：** 库存扣减这一块，你是怎么防止超卖的？Redis 分段消费是什么意思？ ******

业务背景： 营销活动最怕的就是超卖。比如iphone 17只有 10 台，结果发出去 11 台，这属于重大生产事故。同时，如果每次扣减都直接 Update 数据库，高并发下数据库会因为行锁竞争而瘫痪。

我的方案：

1. **Redis 原子预扣减：** 流量进来先扣 Redis，扛住流量。
2. **分段锁（滑块锁）兜底：** 不仅仅是用 `decr`，而是将 `decr` 后的返回值作为索引，配合 `setnx` 加锁。
 - **解释：** 比如库存剩 99，我就锁住 `key_99`。这意味着每一个具体的库存都有唯一的归属，从根本上杜绝了超卖。
3. **延迟队列最终一致：** 扣减成功后，并不急着写库，而是发送消息到延迟队列。由后端的定时任务批

量聚合后，再缓慢更新数据库。

****业务成果：** **避免了超卖现象，大大降低了数据库写的压力

分布式数据与可靠性

****Q：** 针对签到返利这些流程，为什么要解耦？ElasticSearch 在这里起了什么作用？ ******

业务背景： 用户的核心诉求是抽奖快。但抽奖成功后，还有很多后续动作：增加积分、写入流水、发短信通知、运营后台统计数据。如果全部同步执行，接口会很慢；且如果分库分表了，运营后台想查“所有用户的中奖列表”会非常难（涉及到跨库查询）。

我的方案：

- 1. 流程解耦：** 我将“发奖、返利”等非核心动作剥离，通过 MQ 异步处理。
 - 为了保证消息绝对不丢（比如积分一定要到账），我配合了**本地任务表（Task Table）**，实现了“事务性消息”的补偿机制。（详情可看消息队列一章）
- 2. 异构查询：**
 - C 端高并发写：走 MySQL 分库分表（按 UserID 路由）。
 - B 端复杂读：通过 Canal 监听 Binlog，将数据实时同步到 ElasticSearch。运营后台的统计查询直接走 ES。

业务成果： 实现了C 端高性能与B 端多维查询的完美平衡，同时通过 MQ 任务补偿机制，保障了用户资产数据的最终一致性。

追问

Q： 详细讲一下怎么实现O (1) 的抽奖算法

传统的抽奖算法（如轮盘赌算法）通常是遍历奖品列表，累加概率值进行比对，时间复杂度是 $O(N)$ 。当奖品数量多或并发高时，CPU 需要进行大量的循环计算。

而我的抽奖算法的本质是**空间换时间**。我在策略装配阶段，根据奖品概率，预先生成了一个概率查找表存入 Redis。

举个例子

- 假设概率精度是 0.01%（万分之一），我们就创建一个长度为 10,000 的数组（或 Map）。
- 如果奖品 A 的概率是 20%，我们就在这个数组里填入 2000 个奖品 A 的 ID。

- 如果奖品 B 的概率是 1%，我们就在这个数组里填入 100 个奖品 B 的 ID。
- 抽奖时，只需要生成一个 0 到 9999 的随机数作为下标，直接去数组里取值即可。

抽奖时，我只需要生成一个随机数作为下标，直接从 Redis Map 中取值即可。这完全避免了传统算法中 $O(N)$ 的轮询比对，极大提升了吞吐量。

注意

上面这些话术并不是孤立的，比如当面试官问你抽奖流程时，你可以先讲 **场景一（规则树）**，然后顺势带出 **场景三（库存扣减）**：“当然，在规则树的执行过程中，最关键的节点就是库存扣减，为了防止超卖，我特别设计了 Redis 分段锁……”

这样一来一回，项目的考察就占时间了，对于八股的考察也能少点

更新: 2026-01-28 11:37:28

原文: <https://www.yuque.com/u63312805/wnyutc/hcfgsmk38pbd9gva>

设计模式

设计模式

我在京东和美团的面试过程中都被考察过用了什么设计模式

这个问题需要回答 为什么要用以及解决了什么痛点

话术

**** 你的项目里用了哪些设计模式？简单讲讲 ****

在整个大营销抽奖系统中，为了应对复杂的抽奖规则和多变的业务流程，我避免了大量的if-else硬编码，而是通过设计模式来解耦业务逻辑。我核心使用了**责任链模式**处理抽奖前的规则过滤，使用**组合模式（规则树）**处理复杂的库存扣减与兜底逻辑，使用**模板方法模式**定义抽奖的标准全流程，以及**策略模式**封装不同的底层抽奖算法。

责任链模式 —— 抽奖前置规则过滤

不了解责任链的可以看下：<https://refactoringguru.cn/design-patterns/chain-of-responsibility>

- **业务场景：** 在用户参与抽奖之前，我们需要进行一系列的资格判断：
 - a. 用户是否在**黑名单**中？（如果在，直接拦截返回固定小奖）。
 - b. 用户是否触发了**权重规则**？（例如积分消耗达到一定额度，必中某个范围的奖品）。
 - c. 如果都不满足，则走**默认抽奖**逻辑。这些规则是**互斥且有顺序**的。
- **我的实现：**
 - 我定义了 `ILogicChain` 接口，实现了 `BlackListLogicChain`（黑名单）、`RuleWeightLogicChain`（权重）、`DefaultLogicChain`（默认）等节点。
 - 通过 `next` 指针将它们串联成一条链。请求进入时，如果当前节点能处理（如命中黑名单），则直接返回结果；否则将请求传递给下一个节点。
- **收益：**
 - **解耦：** 将黑名单、权重等逻辑拆分到独立的类中，避免了 Service 层出现几百行的 `if-else`。
 - **易扩展：** 如果以后要加一个白名单或者VIP规则，我只需要新增一个 Chain 节点并调整装配顺序。

序，完全不需要修改核心代码。

组合模式/规则树 —— 抽奖中/后置决策引擎

不了解组合模式的可以看下：<https://refactoringguru.cn/design-patterns/composite>

- **业务场景：** 当用户抽中某个奖品 ID 后，并不代表他真的拿到了奖。我们需要进行更复杂的多叉判断：
 - 先判断**次数锁**（用户今天抽了几次？是否满足该奖品的解锁次数？）。
 - 再判断**库存**（Redis 库存够不够？）。
 - 如果库存不够，是直接报错，还是走**兜底逻辑**（降级为 1 积分）？这就构成了一棵**决策树**。
- **我的实现：**
 - 我采用了**组合模式**的思想构建了一棵规则树。
 - 定义了 `RuleTreeNode`（树节点）和 `RuleTreeNodeLine`（连线/边）。
 - 每个节点（如 `RuleStockLogicTreeNode`）负责执行具体的逻辑（扣减库存）。
 - 执行完后，根据结果（True/False）顺着连线找到下一个节点继续执行。
- **收益：**
 - **可视化/动态化：** 这种树状结构可以很方便地映射到数据库配置中。产品经理可以在后台配置A 奖品库存不够 -> 走B兜底，而研发不需要改代码，只需要通过引擎（`DecisionTreeEngine`）动态执行即可。

模板方法模式 —— 定义抽奖标准骨架

- **业务场景：** 整个抽奖流程是非常标准的：参数校验 -> 获取策略 -> 执行责任链（前置规则） -> 执行规则树（后置处理） -> 包装结果。这个流程骨架在不同的抽奖策略中是通用的，但具体的细节可能不同。
- **我的实现：**
 - 我在 `AbstractRaffleStrategy` 抽象类中定义了一个 `performRaffle` 方法，规定了上述的标准步骤。
 - 具体的子类（如 `DefaultRaffleStrategy`）只需要关注具体的细节实现，或者通过钩子方法进行微调。
- **收益：**
 - **规范流程：** 保证了所有抽奖入口都遵循严格的逻辑闭环，防止某个新来的开发写漏了前置校验或后置记录。

策略模式—— 底层算法封装

- **业务场景：**抽奖的核心是根据概率随机命中。但是针对不同的奖品配置量级，算法可能不同。
- **我的实现：**
 - 我定义了 `IAlgorithm` 接口。
 - 针对大多数场景，实现了 $O(1)$ 算法（`O1Algorithm`）：利用 Redis Map 预生成的查找表，空间换时间，性能极高。
 - 针对极低概率或超大范围场景，保留了 $O(\log N)$ 算法（`OLogNAlgorithm`）：使用二分查找。
 - 在 `StrategyArmory`（策略装配库）中，根据配置动态选择使用哪种算法。

追问

** 为什么责任链要用“互斥”的设计？一般的责任链不是所有节点都跑一遍吗 **

责任链有两种形态。一种是**流水线模式**，所有节点都处理；另一种是**处理者模式**（审批流），只要有一个人审批通过（处理）了，流程就结束。在抽奖前的规则中，如果一个用户已经在黑名单里被拦截了，他就不应该再有资格去计算权重规则了。所以我的设计是：一旦某个节点处理了（返回了奖品），后续节点不再执行。

** 规则树和责任链有什么区别？为什么不都用责任链？ **

- 责任链适合线性的、单向的逻辑，比如 $A \rightarrow B \rightarrow C \rightarrow D$ 。
- 规则树适合分叉的、多维的逻辑。比如 A 成功走 B，A 失败走 C；B 成功走 D，B 失败走 E。库存扣减后涉及复杂的兜底分支（库存不够变积分，积分不够变谢谢惠顾），这是一种树状决策，用责任链很难表达这种分支跳转，所以选了规则树。

更新: 2026-01-28 11:37:20

原文: <https://www.yuque.com/u63312805/wnyutc/lsr95ma2geenychx>

数据结构

数据结构

可能一面会问这种问题：

“你的项目里面用了哪些数据结构，简单讲一下”

这个比较简单，但需要你整理好，在这样的场景中，我一般会罗列吟唱一下：用了hashmap，set，linkedList，.....，其中主要用hashmap，主要用来.....

话术

**** 你的项目里面用了哪些数据结构，简单讲一下？ ****

在整个大营销项目中，除了基础的 ArrayList 用于数据传输和 HashMap 用于配置装配外，我最核心的业务逻辑其实是建立在三种特定的数据结构之上的：

1. 散列表 (Hash Table / Map)：用于实现 $O(1)$ 复杂度的抽奖算法。
2. 链表 (Linked List)：用于构建责任链模式，处理抽奖前的规则过滤。
3. 树 (Tree)：用于构建规则引擎 (决策树)，处理复杂的库存与兜底逻辑。

散列表/哈希表 (HashMap) —— 核心场景： $O(1)$ 抽奖策略装配

- 怎么用的：在我的抽奖策略装配 (StrategyArmory) 模块中，我没有使用简单的 List 遍历来判定中奖 (那是 $O(N)$ 的)。
- 具体实现：我利用 HashMap (或者说数组结构) 构建了一个“概率查找表”。
 - 我将所有奖品的概率按比例展开 (例如 0.01% 对应 1 个位置，20% 对应 2000 个位置)，存入一个大小为 10000 的 Map 中 (`Map<Integer, Integer>`)。
 - Key 是 1 到 10000 的索引，Value 是奖品 ID。
- 收益：这样抽奖时，我只需要生成一个随机数 `randomKey`，就能通过 `map.get(randomKey)` 在 $O(1)$ 的时间复杂度内拿到结果，完全避免了高并发下的 CPU 循环计算消耗。

链表 (Linked List) —— 核心场景：责任链模式 (抽奖前置规则)

- 怎么用的：在抽奖的“前置过滤”阶段 (判断黑名单 -> 权重规则 -> 默认抽奖)，我使用了单向链

表结构。

- **具体实现：** 虽然代码中没有直接使用 `LinkedList` 类，但我的 `AbstractLogicChain` 抽象类中定义了 `protected ILogicChain next` 属性。
 - 每个逻辑节点都持有下一个节点的引用，这在逻辑上就是一个标准的**单向链表**。
- **收益：** 这种结构让我的业务逻辑非常灵活。如果我想插入一个新的规则（比如“白名单”），我只需要调整链表的指针连接顺序，而不需要修改核心代码。

树 (Tree) —— 核心场景：规则引擎决策树（抽奖中后置规则）

- **怎么用的：** 这是我项目中逻辑最复杂的“库存扣减与兜底”环节。我没有写死 `if-else`，而是设计了一颗**二叉/多叉决策树**。
- **具体实现：** 我定义了 `RuleTreeNode`（树节点）和 `RuleTreeNodeLine`（边/连线）。
 - 节点代表逻辑（如：库存扣减节点、次数锁节点）。
 - 连线代表判断条件（如：库存充足走左边，库存不足走右边）。
- **收益：** 引擎（`DecisionTreeEngine`）通过遍历这棵树来执行逻辑。这使得业务逻辑是可视化的、可动态配置的。

队列 (Queue) —— 核心场景：异步消息与延迟队列

- **怎么用的：** 在系统架构层面，我大量使用了队列。
- **具体实现：**
 - **Redis 延迟队列 (RBlockingQueue)：** 用于库存扣减后的异步同步。
 - **RabbitMQ 消息队列：** 用于发奖解耦。
- **收益：** 利用队列的FIFO特性，实现了流量的削峰填谷，保证了请求的有序处理。

追问

**** 你提到的 O(1) 算法，如果 Map 很大（比如 100万 key）会不会占用内存 ****

会的。所以我在设计时做了优化。对于极低概率（如百万分之一）的奖品，我不会真存 100 万个 Key，而是采用两段式结构。在 Map 中只存一个占位符，命中后再进行二次随机。这其实是以时间换空间的平衡

**** 为什么规则引擎要用树，不用简单的 List 顺序执行 ****

因为业务逻辑不是线性的。比如库存不足时，我需要跳过后续所有步骤直接走兜底，这是一个分支逻辑。List 适合线性逻辑，而 Tree 天然适合分支判断逻辑。

更新: 2026-01-28 11:36:45

原文: <https://www.yuque.com/u63312805/wnyutc/wzz4tsd47layqg9g>

消息队列

消息队列

MQ（消息队列）是高并发系统中仅次于缓存的核心组件，也是我在面试过程中项目被问到比较多的内容。

对于中间件，我的思路是，要把场景+模式+收益讲清楚，因为中间件千变万化，选型很多，因此场景是什么，选了什么，收益是什么，才是面试官最想听到的内容。

所以我在这块准备的一个核心问题是：你的项目中哪里用到了MQ？为什么要用？

话术

Q：你的项目中哪里用到了MQ？为什么要用？

****背景：****在大营销平台中，抽奖是一个高并发、长链路的业务。用户的每一次点击，不仅涉及“资格校验、策略计算、库存扣减”，抽中奖后还涉及“奖品发放（如充值积分）、中奖通知、写入流水”等后续操作。如果这些操作全部同步执行，不仅接口响应慢（RT高），而且一旦数据库瞬间压力过大，整个系统都会雪崩。

难点/痛点：

1. ****高并发写库压力：****几万人在一秒内抽奖，如果每次扣库存都直接Update数据库，数据库很容易被打崩。
2. ****核心流程被拖累：****用户只关心“有没有中奖”，不关心“积分是不是立刻到账”。如果因为发奖服务（第三方接口）超时导致用户抽奖失败，体验极差。
3. ****分布式事务难题：****既要保证数据库记录写入成功，又要保证MQ消息一定发送成功（不能抽中奖了却没发货）。

方案（我的MQ三大应用场景）：

第一：削峰填谷（解决库存高并发难题）

- ****场景：****抽奖时的活动库存扣减
- **做法：**我设计了Redis预扣减+延迟队列异步同步的方案。

- 用户请求先在Redis中 `decr` 扣减库存，只要Redis扣减成功，就视为抢购成功。
- 然后，我将“扣减库存”的动作发送到MQ（或者是Redis延迟队列），由后端的定时任务（`UpdateActivitySkuStockJob`）慢慢消费，批量更新数据库。（往延迟队列里添加是每3s加一个，job任务是每5秒取一个）
- **收益：**将数据库的TPS压力转移到了Redis，保护了数据库，同时抗住了秒杀级的流量。

第二：异步解耦（提升核心链路性能）

- **场景：**用户中奖后的“发奖”环节。
- **做法：**在 `Strategy` 领域计算出奖品ID后，系统只需记录“用户中奖订单”，然后直接通过MQ发送一条 `SendAwardMessageEvent` 消息，立刻给前端返回“中奖成功”。
- **消费者：**`SendAwardCustomer` 监听到消息后，再异步去调用积分服务充值或发放优惠券。
- **收益：**抽奖接口的RT（响应时间）从几百毫秒压缩到几十毫秒，且发奖服务的故障不会阻塞主流程。

第三：分布式一致性保障（任务补偿机制）

- **场景：**确保“中奖记录入库”和“发奖消息发送”这两个动作要么都成功，要么都有补偿（最终一致性）。
- **做法：**采用了“本地消息表（TaskTable）”+“事务性发件箱模式”。
 - 在写入“中奖订单”的同一个数据库事务中，我写入了一条 `Task` 记录（状态为init）。
 - 事务提交后，再发送MQ消息。如果MQ发送失败，后台会有定时任务扫描 `Task` 表重新发送，确保消息至少被消费一次。

效果：通过引入RabbitMQ，我实现了核心链路的轻量化。压测显示，在单机1000QPS下，数据库CPU占用率降低了50%以上。

追问

Q：你用到了MQ，怎么处理消息重复消费的问题（幂等性）？

在我的架构中，幂等性是在消费端处理的。比如发奖环节，我的消息体里包含唯一的 `orderId`（订单号）。消费者在发奖前，会先用 `orderId` 去查询/通过数据库唯一索引判断“该订单是否已经发过奖”。如果发过，直接ACK确认并丢弃消息；如果没发过，再执行发奖逻辑。这保证了即使MQ重发消息，用户也不会收到两次积分。

Q：为什么发奖要用MQ？直接用线程池异步执行不行吗？

1. 线程池虽然也能异步，但它是**内存级**的。如果服务器突然宕机，线程池里的任务就全部丢失了，用户就的奖品就丢失了。而MQ具有**持久化能力**。即使服务重启，消息依然在队列中，重启后可以继续消费。对于“资产发放”这么敏感的业务，**可靠性比单纯的性能更重要**，所以我选择了MQ。
2. 除了持久化防丢失外，MQ 还有一个核心优势是**保护下游**。如果使用线程池，当上游请求暴增时，线程池队列瞬间堆满，会占用大量 JVM 内存甚至导致内存溢出（OOM），或者**因为拒绝策略丢弃任务**。而MQ作为一个独立的中间件，可以堆积千万级的消息。无论上游流量多大，下游（发奖服务）都可以按照自己的速率慢慢消费，不会被流量洪峰冲垮

Q：你刚才提到了库存是异步扣减的，那如果Redis扣到0了，数据库里还有库存，或者队列里还有堆积的消息没消费，怎么保证数据一致？

- ****正常流程：****Redis扣1个，延迟队列慢慢同步1个
- **临界流程（库存为0）：**当Redis中的库存瞬间被扣减为0时，我会立即发送一条高优先级的“库存清空消息”。
- ****强制同步：****消费者收到这条消息后，会做两件事：
 - 直接将数据库的库存强制更新为0（兜底）。
 - 清空/忽略延迟队列中剩余的库存扣减消息。

更新: 2026-01-28 11:36:37

原文: <https://www.yuque.com/u63312805/wnyutc/xa29r3lnfiup6bpg>

压测与瓶颈分析

压测与瓶颈分析

有些小伙伴没有压测的条件，给大家一些参考

同时，大厂实习也需要学怎么用prometheus+Grafana去检测系统的运行情况，甚至会让你用jmeter去做压测

如果面试的过程中突出你有对你项目的压测经验，会给面试官留下你是认认真真对待一个项目，会去考虑软件生命整个周期，而不是简单做完就结束了。

因此，对于进阶的小伙伴，强烈推荐大家自己对自己的项目做下压测，实在没时间没条件了，也可以参考一下我的话术，把自己的技术栈补全。

对于这块，需要说清楚：

自己是怎么做压测的、压测的结果是什么、瓶颈是什么、怎么改进（可只说思路）、改进后的效果（可无）

话术

Q：你的项目有没有做压测，是怎么做的呢？

一句话版本：在压测环节，我基于 **3台 4C8G 服务器** 搭建了模拟生产环境，使用 JMeter 进行**梯度压测**。起初为了在**Docker 容器化部署**下，并发达到 200 时系统出现性能拐点，**TPS 卡在 150**，**响应时间飙升至 1.5秒**。通过监控分析，我定位到瓶颈在于 **Docker 网络 I/O 损耗**、**Tomcat 默认 200 线程饱和** 以及 **数据库连接池过小**。针对这些问题，我实施了**去容器化 + 中间件连接池扩容**（Tomcat 调至 400/MySQL 调至 2000）+ JVM 内存调优的组合优化方案。最终压测结果显示，**TPS 翻倍提升至 308**，**TP99 稳定在 400ms 左右**，成功消除了基础设施层面的短板，将性能瓶颈转移到了业务逻辑锁竞争上。

压测环境与策略 （重要，一定要记住，如果要用的话）

- **硬件**：使用阿里云 ECS 搭建了 3 台 4C8G 的服务器。为了模拟真实的分布式环境，我将应用服务、中间件（Redis/MySQL/ZK）、监控系统（Prometheus+Grafana）分开部署。
- **部署方式**：初期为了交付效率，全量采用 **Docker 容器化部署**。
- **压测策略**：使用 **JMeter** 作为压力机，采用**梯度压测法**。
- 并发线程数从 10 开始，按 10、50、100、200、300 逐步递增。
- 重点关注指标：TPS (吞吐量)、RT (响应时间)、TP99、CPU/内存利用率、Tomcat 活跃线程数、数据库连接池状态。

初次压测结果和分析

数据：在并发线程数达到 200 时，系统出现了明显的性能拐点：

- **TPS 瓶颈**：最高卡在 150 TPS 左右，无法继续上升。
- **RT 飙升**：接口响应时间从 100ms 飙升至 1.5秒 以上。
- **资源表现**：
 - **应用服务器**：CPU 利用率仅 50% 左右，负载较低（Load < 4），说明 CPU 不是瓶颈。
 - **内存**：堆内存使用仅 600MB+，但 **Minor GC 非常频繁**（1.5次/秒）。
 - **Tomcat**：监控显示 **Tomcat 繁忙线程数** 达到默认最大值 200，且出现排队现象。
 - **数据库**：MySQL 连接数卡在默认值，中间件服务器负载较高。

问题诊断：通过分析Grafana，我定位到三个核心瓶颈：

1. **容器网络开销**：Docker 部署模式下，I/O 密集型组件（MySQL/Redis）存在网络桥接性能损耗。
2. **Tomcat 线程池饱和**：默认 200 个线程无法处理 200+ 的并发请求，导致请求在队列中堆积，RT 变长。
3. **连接池“双重限制”**：
 - **DB 端**：MySQL 默认最大连接数仅 150。
 - **App 端**：HikariCP 连接池默认仅 10-25，导致大量请求在等待获取数据库连接。

性能调优方案

针对上述问题，我实施了基础设施+中间件+JVM 三个方向的优化

- **架构层**：
 - 将 I/O 敏感的 **MySQL、Redis、Zookeeper** 以及**应用服务**从 Docker 迁移至**物理机/宿主机直接部署**，消除 Docker NAT 网络的性能损耗。
- **中间件层（扩充连接池）**：

- Tomcat: 将 `max-threads` (最大线程数) 从 200 提升至 400, 并配合调整 `accept-count` (等待队列) 至 1000, 提升吞吐能力。
- MySQL: 将 `max_connections` 调整至 2000, 防止服务端拒绝连接。
- HikariCP / Redis Pool: 同步调大应用端的数据库连接池和 Redis 连接池参数, 避免应用内部争抢连接。
- JVM 层 (内存优化) :
 - 发现初始堆内存分配过小导致 GC 频繁, 将堆内存扩大至 4 G (`Xmx4g`) 。
 - 调整年轻代比例, 优化 `Survivor` 区设置, 减少对象过早进入老年代引发的 Full GC 风险。

追问

Q: 怎么发现是docker带来的性能损耗

首先观察到应用服务器 CPU 不高但响应很慢, 且中间件服务器 Load 异常高, JMeter 伴有 `socket closed` 错误。我们怀疑是高频 Redis 交互在 Docker Bridge 模式下产生了巨大的 NAT 转发开销。为了验证, 我做了一个**对照实验**: 保持代码和配置不变, 仅将 MySQL 和 Redis 从容器迁移到物理机直连。结果 TPS 瞬间从 150 提升到 268, 从而证实了 Docker 虚拟网络是核心瓶颈之一。(一开始用 docker的原因是为了图方便)

****Q: 为什么 CPU 60% 就上不去了? ****

这说明系统不再是单纯的计算密集型瓶颈, 而是I/O 密集型或锁竞争。既然 Tomcat 线程和 DB 连接都够了, 可能是Redis 分布式锁的争抢导致线程阻塞, 或者是网络带宽限制。如果还要提升, 我会考虑优化锁粒度 (分段锁) 或引入本地缓存。

Q: 为什么 Docker 部署会影响 MySQL 性能?

Docker 默认的 Bridge 网络模式涉及 NAT (网络地址转换), 在高并发大量小数据包传输时 (如 Redis/MySQL 交互), CPU 会消耗在网络中断和包转发上, 增加延迟。对于数据库这种重 I/O 应用, 生产环境通常建议使用 Host 模式或物理机部署。

**** Q: Tomcat 线程数是越大越好吗? ****

不是。线程过多会导致 CPU 频繁进行上下文切换 (Context Switch), 反而降低性能。400 是根据 4C8G 的配置和压测结果摸索出的经验值, 通常是 CPU 核数的几十倍 (I/O 密集型), 需要通过压测找

到“吞吐量”和“上下文切换”的平衡点。

更新: 2026-01-28 11:45:26

原文: <https://www.yuque.com/u63312805/wnyutc/akobmq7032vtg22u>

DDD

DDD

关于DDD，核心的问题是，面试官觉得你为什么会选择这么DDD而不是MVC，因此我们需要准备好相关的回答，需要从几个角度讲：

1. DDD的基础概念
 2. 与MVC的对比
 3. 在项目中是怎么使用DDD的，怎么划分领域，举个例子
-

话术

Q：你在项目中是怎么使用DDD的，为什么要用DDD

一段话描述：

关于为什么要用 DDD，其实是因为大营销平台的业务链路实在是太长、太复杂了。

在做这个项目初期，如果按照传统的 MVC 三层架构去写，所有的逻辑——比如用户能不能抽奖、黑名单怎么校验、概率怎么算、库存怎么扣、奖品怎么发——全部都会堆积在一个 service 里。

这会导致这个 Service 类变成一个几千行的类^{**}。^{**} 这样逻辑会高度耦合。举个例子，可能我只是想调整一下抽奖的概率算法，结果因为代码纠缠在一起，一不小心就把库存扣减的事务逻辑给搞崩了。而且大部分代码都是简单的 Get/Set 的**贫血模型**，业务逻辑和数据库操作绑死在一起，维护起来非常痛苦

为了解决这个问题，我参考了 DDD 的思想，把整个抽奖流程按照**业务边界**拆分成了三个核心的领域，实现了**计算与事务的分离**。

我是这样划分的：

第一个是【策略领域（Strategy）】，它解决的核心问题是‘怎么抽’。您可以把它理解为纯粹的**计算引擎**。它不关心用户是谁，也不关心活动有没有过期，它只负责接收一个策略 ID，然后通过**责任链模式**处理黑名单、权重过滤，再通过**规则树**处理库存锁和兜底逻辑，最后计算出一个奖品 ID。这就实现了**关注点分离**，这个领域里全是高密度的计算逻辑，不掺杂任何用户状态。

第二个是【活动领域（Activity）】，它解决的问题是‘能不能抽’。这是**事务发生的核心地带**，也是整个系统的‘看门人’。它负责管理用户的参与生命周期，比如校验活动状态、扣减用户个人的日/月限额，以及调用 Redis 进行库存的预扣减。它保证了用户不能超领，也不能在活动未开始时参与。

第三个是【奖品发放领域（Award）】，它解决的问题是‘发什么’。当策略领域算好结果后，发奖领域只负责最后的交付。我在这里用了**工厂模式**，因为奖品类型很多，可能是发积分、发优惠券码，或者是调用第三方 RPC 充值。这个领域把具体的交付细节和核心抽奖逻辑完全解耦了。”

通过这种划分，我觉得最大的收益有两点：

第一是**高内聚、低耦合**。现在的架构下，如果运营想要修改抽奖的概率算法，我只需要去改动 Strategy 领域的代码，完全不用担心会影响到 Activity 领域的库存扣减逻辑，系统的稳定性大大提高了。

第二是**防腐与纯粹性**。我在设计时遵循了**依赖倒置原则**。领域层只定义接口，由基础设施层去实现接口。这样我的领域层操作的都是**充血模型**的实体对象，完全不知道数据库的存在，彻底解决了持久化对象（PO）入侵业务逻辑的问题，让核心业务代码非常干净。

^{**}背景：^{**}大营销平台的核心业务链路非常长，涵盖了用户参与资格校验、执行抽奖算法、奖品发放、积分返利等多个复杂环节。在传统的MVC三层架构中，如果不加以控制，所有的业务逻辑（比如判断黑名单、计算库存、随机算法、发奖）都会堆积在Service层，导致Service类极其臃肿，难以维护，且业务逻辑与数据库操作高度耦合（贫血模型）。

难点/痛点：最大的难点在于规则的多变性和业务的独立性。比如，抽奖策略可能包含黑名单、权重、规则树等多种玩法，如果它和活动库存管理耦合在一起，修改一个概率计算逻辑可能会误触到库存扣减的代码。我们需要一种架构，能让核心的计算逻辑与事务逻辑分离，防止业务逻辑随着需求迭代而腐化。

****方案（我的划分逻辑）：** **基于DDD的思想，我按照业务边界上下文将系统拆分为三个核心领域和几个支撑领域：

1. 策略领域—怎么抽

- **职责：**这是整个系统的核心计算引擎。它不关心用户是谁，也不关心活动是否过期，只负责接收策略ID，通过责任链模式（处理黑名单、权重）和规则树（处理库存锁、兜底），计算出最终的奖品ID。
- **特点：**这里大量使用了充血模型，比如 `StrategyEntity` 内部封装了获取策略规则的方法，而不是单纯的数据结构。

2. 活动领域—能不能抽

- **职责：**负责管理用户的参与生命周期。包括校验活动状态、扣减个人日/月/总额度、活动库存的预扣减（Redis）、以及生成抽奖单。
- **特点：**这是事务发生的核心地带，保证用户不能超领、不能在活动未开始时参与。

3. 奖品发放领域—发什么

- **职责：**当策略领域算出来结果后，由该领域负责具体的交付。比如发放积分、发送优惠券码、或者调用第三方RPC接口充值。
- **特点：**通过工厂模式适配不同的发奖逻辑，与核心抽奖链路解耦。

*（可选补充：此外还有_**Rebate（返利领域）****负责签到等行为的聚合，**Credit（积分领域）_负责账户余额变动。）*

****效果：** **通过这种划分，我实现了计算与事务的分离：

1. **高内聚：**如果未来要调整抽奖概率算法，我只需要改动Strategy领域，完全不需要担心影响到Activity领域的库存扣减逻辑。
2. **防腐：**通过依赖倒置，领域层定义接口（RepositoryInterface），基础设施层实现接口。领域层只操作纯粹的实体对象，完全不知道数据库的存在，彻底解决了持久化对象（PO）入侵业务的问题。

追问

Q：你提到了DDD架构能让“计算”与“事务”分离，Strategy 领域完全不操作数据库吗？

Strategy 领域在执行过程中主要依赖 Redis 里的缓存数据（如概率表、规则树结构）进行纯内存计算，追求极高的性能。只有在构建规则树或者责任链初始化时，才会通过 Repository 接口去读取配置。它计算出的结果是一个单纯的 `AwardEntity`，不包含写入数据库的操作。写入操作是由应用层协调，在 Activity 领域生成订单时才落库的。

****Q：为什么要单独拆一个 Award 领域？发奖不就是一个 Service 方法吗？ ****

一开始我也觉得简单，但后来发现发奖的易变性很高。今天发的是积分，明天可能是虚拟币，后天是实物物流单。如果写在 Activity Service 里，每次加新奖品类型都要改核心代码。拆分后，Award 领域专注“交付”，并且可以通过 MQ 异步解耦，保证核心抽奖接口的响应速度（RT）

Q：DDD的充血模型体现在哪里

以策略实体为例，在贫血模型中，我们通常会把“计算下一个节点”的逻辑写在 Service 里。而在我的项目中，`RuleTreeNodeEntity`（规则树节点实体）自己就包含了 `process` 行为，它知道如何根据当前的输入值，判断是走左子树还是右子树。Service 只需要调用 `entity.process()`，而不需要关心具体的判断逻辑。

Q：在你的项目里面领域之间是怎么交互的

遵循 DDD 原则，领域之间不能直接互相调用（避免循环依赖）。我是通过应用层进行编排的。例如：`RaffleActivityController` -> `RaffleActivityService`（应用层）-> 先调用 `ActivityDomain` 校验资格 -> 再调用 `StrategyDomain` 执行抽奖 -> 最后调用 `AwardDomain` 发放奖品。

Q：实体与值对象的区别是什么，在项目里有做区分吗

在模型设计细节上，我严格区分了实体和值对象。

- **实体（Entity）**：像奖品、订单、用户账户，它们有唯一 ID，状态会随生命周期改变（比如订单从待发货变成已发货）。
- **值对象（VO）**：像抽奖策略里的规则树结构 `RuleTreeVO`、规则权重配’。它们描述的是具体的属性值，没有唯一生命周期，仅仅用来承载数据。

Q：聚合根有了解过吗，项目里面是怎么用的

根据 DDD 原则，一个聚合就是一个事务的一致性边界。比如在活动领域，我定义了“用户参与行为”为一个聚合，其中用户活动账户实体和用户活动订单实体都在同一个聚合内。当用户参与抽奖时，扣减【个

人额度】和写入【抽奖单】必须在同一个数据库事务中成功或失败。如果我把它拆得太散，会导致分布式事务问题。通过聚合根的设计，我确保了核心业务操作的强一致性。

更新: 2026-01-28 11:36:30

原文: <https://www.yuque.com/u63312805/wnyutc/uod9m8i5b3ufot10>

redis

redis

redis是项目里面的重中之重，基本90%的大营销面试，都会问到redis相关的问题

对于redis，我们需要准备的核心问题，是除了缓存之外redis做了什么，如果让你讲redis，只是平铺直叙“Redis快是因为基于内存”的这种废话，那么完全不能体现你的学习能力，项目也白做了

话术

背景：

在我的大营销平台中，核心的‘抽奖策略引擎’承载着整个系统的最高并发流量。它的业务逻辑非常复杂，一次用户的抽奖请求，需要经过‘黑白名单校验->权重规则计算->随机数算法->奖品库存扣减’等一连串同步操作。

作为一个面向C端用户的秒杀级系统，我们面临着三个大的挑战：

1. 高并发：活动开启瞬间，QPS可能会瞬间飙升至万级，数据库根本无法直接承载。
2. 高计算：抽奖不仅仅是查数据，还涉及复杂的概率算法（比如在一万个商品中随机抽取一个），对CPU资源消耗极大。
3. 高一致性：营销活动涉及到真实的资产（如现金红包、iPhone），绝不允许超卖是系统的底线

难点/痛点：

1. **行锁灾难**：在传统的数据库架构中，库存扣减通常是`update table set stock = stock - 1 where id = xxx`。当一万个人同时抢一个热门奖品时，这一万个请求会同时去争抢数据库中的同一把行锁。这会导致数据库连接池瞬间被占满，线程大量阻塞，RT（响应时间）从几毫秒恶化到几秒，甚至直接拖垮整个数据库，引发服务雪崩。
2. ****抽奖业务复杂概率计算的CPU瓶颈****：我们的抽奖策略支持动态配置，比如某个奖品的概率是0.001%，另一个是20%。如果每次请求进来，都在内存中遍历所有奖品去计算累加概率（O(N)复杂度），在并发量大时，CPU会大量消耗在循环计算上。我们需要一种方案，能在O(1)的时间复杂度内直接定位到中奖结果，用‘空间换时间’来释放CPU压力。
3. ****需要有更细粒度的锁方案****：使用Redis的`decr`虽然可以原子扣减库存，但它有一个缺陷：它无法记录‘是谁扣减了库存’。在分布式网络中，如果发生超时重试，简单的`decr`可能会导致一个用户扣减

了两次库存（数据不一致）。我们需要一种机制，既能享受Redis的高性能，又能像数据库一样拥有颗粒度极细的锁竞争能力，确保每一次扣减都是可追溯、可防重的。”

Q：你的项目中哪里用到了Redis？除了做缓存还能做什么？

在我的大营销项目中，Redis不仅仅用来缓存，我主要在以下三个核心场景中深度使用了Redis

1. 策略装配与O(1)抽奖算法

背景：抽奖策略通常很复杂，包含几十个奖品、不同的概率（0.01%到50%）、以及各种规则权重。如果每次用户请求都在CPU内存中实时计算随机数和概率区间，在高并发下CPU会成为瓶颈。

痛点：传统的遍历比对算法复杂度是O(N)或O(logN)，且难以动态调整。

方案：我利用Redis实现了概率查找表。在活动预热阶段，我将概率拆解为整数（如0.01%拆解为10000个位置中的1个）。生成一个巨大的Map，Key是StrategyId_RateKey，Value是乱序排列后的奖品ID表。抽奖时，只需生成一个随机索引，直接从Redis的Map中get出来即可。

收益：将抽奖算法的复杂度从O(N)降低到了严格的O(1)。无论奖品配置多少个，抽奖耗时永远固定在一次RedisIO的时间（约1ms）。

2. 基于滑块锁的库存扣减

****背景：****秒杀场景下的库存扣减

****痛点：****简单的 `decr` 虽然原子，但在网络抖动重试时无法保证幂等，且无法追溯“是谁扣走了第99个库存

****方案：****我设计了Decr原子扣减+SetNx滑块锁的双重机制

- 先执行 `decr`，假设返回值是99。
- 使用 `setNx` 加锁，Key是 `strategy_id_award_id_99`。
- 这相当于给每一个具体的“库存槽位”加了一把独占锁。只有抢到“第99号”令牌的请求，才被允许真正获得奖品。

****收益：****这种设计比全局分布式锁性能高出数量级，因为它锁的是具体的某一个商品，而不是锁住整个商品表

3. 分布式Session与幂等令牌

****背景：****用户行为限制与防重

方案：

1. 活动额度管理：使用Redis的HashMap结构存储用户的(UserId_ActivityId->Count)，利用incr原子递增来判断用户是否超过日限额/月限额。
2. 业务幂等：在MQ消费和发奖环节，使用Redis存储OrderId作为防重Key，防止分布式场景下的重复

发奖。

追问

Q: 你刚才提到的“概率查找表”，如果概率是 0.0001%，那 Redis 里不是要存 100 万个 Key？会不会是大 Key

1. **分段存储**：我的算法支持配置。对于超低概率，我不会傻傻地存100万个 Key。我采用了分层的策略：高概率奖品使用 O(1) 查找表；而对于那种极低概率的大奖，我在查找表中只存一个占位符，命中占位符后，再进行二次随机计算。
2. **数据结构优化**：我使用的是 Redis 的 `Map` 结构（Hash），而不是巨大的 String。抽奖时，我只需要 `HGET key field`，获取其中某一个位置的值。Redis 对于 Hash 的取出操作非常高效，不会把整个大Map读取到内存中，所以网络开销很小。

Q: 你的滑块锁如果 `decr` 成功了，但是 `setNx` 失败了（比如 Redis 挂了），会怎么样？

这种情况会被视为扣减失败**。在我的代码逻辑中，`decr` 只是预操作。只有当 `setNx` 返回 `true`，才算真正锁定成功。如果 `setNx` 失败（比如因为网络超时），业务层会捕获异常，视为没抢到。**

后果：可能会导致少卖，但绝对不会超卖。对于营销活动来说，不少卖是优化的方向，不超卖是底线。后续可以通过库存回滚任务扫描未被关联订单的 Key 进行恢复

** Q: Redis 宕机了怎么办？整个抽奖就挂了吗？ **

1. **本地缓存（Guava）**：对于“策略配置、规则树结构”这种读多写少的数据，我在应用层有一层 Guava本地缓存。Redis 挂了，短期内可以读本地内存继续提供读服务。
2. **兜底机制**：对于“库存扣减”这种强依赖 Redis 的写操作，如果 Redis 连不上，代码中的规则树会捕获异常，自动降级走向“兜底节点”（只发积分或谢谢惠顾）。这样用户端看到的是“未中奖”，而不是“系统错误 500”，保住了用户体验。

Q: 你的抽奖算法里，如果是万人抢一个热门奖品（比如 iPhone），那个奖品的库存 Key 会不会成为热点 Key？怎么解决

我的项目中，我采用了 滑块锁 + Key 散列 的机制来解决这个问题

1. 分散流量：我的库存扣减不是锁一个 `stock_count` 变量，而是利用 `decr` 后的返回值作为逻辑分片。比如库存有 100 个，我就相当于有 100 个独立的锁 Key (`strategy_id_award_id_100, ..._99, ..._98`)。
2. 效果：用户的并发请求虽然是针对同一个奖品，但在 Redis 层面，它们争抢的是不同的 Key（不同的滑块位置）。虽然 `decr` 操作本身还是针对同一个 Key，但 Redis 的 `decr` 性能极高（纯内存操作）。真正的锁竞争压力（`setNx`）被分散到了 100 个不同的 Key 上。
3. 进一步优化（假如量特别大）：如果 `decr` 的那个 Key 依然扛不住，我会考虑将库存物理拆分。比如将 1000 个库存拆分为 `stock_part_1` 到 `stock_part_10`，每份 100 个，随机将用户路由到不同的库存桶中进行扣减。

Q：为什么一定要加分段锁（Key+Index）？直接 Redis `decr/incr` 不行吗？`decr` 也是原子的，只用 `decr` 会有什么致命问题？

表面上看，`decr` 确实能保证计数器的原子递减，但在分布式业务链路中，单纯的 `decr` 存在库存状态不可追溯的致命缺陷

我不直接用 `decr` 的核心原因有两点：

1. 资源独占标记（解决‘谁拿了库存’的问题）：

- 单纯 `decr` 只是把数字从 100 变成了 99。如果此时服务宕机、或者 MQ 发送失败，数据库没有生成订单。
- 后果：只有 Redis 里的数字变少了，但我们完全不知道这少掉的 1 个库存是被谁（哪个用户、哪个请求）拿走的。这会导致库存凭空消失（少卖），且无法通过日志恢复。
- 我的方案（分段锁）：`decr` 返回 99 后，我立刻用 `setNx(strategy_id_award_id_99, userId)` 加锁。
- 收益：如果出现异常，运营人员或补偿脚本可以通过扫描 `key_99`，知道“哦，原来是 UserA 拿走了第 99 号库存”，从而决定是补发奖品还是回滚库存。

2. 并发竞争的颗粒度控制：

- 虽然 `decr` 是原子的，但后续的发奖逻辑（写库、发 MQ）不是原子的。
- 通过 `setNx` 锁定具体的库存索引，我实际上实现了一个细粒度的乐观锁。它确保了在同一时刻，针对第X个商品的处理是唯一的。这比单纯依赖计数器更能保障复杂业务链路（如包含第三方接口调用）的一致性。”

Q：用 SETNX 加锁兜底的话，在高并发秒杀场景下，加锁会让性能退化成串行，怎么保证 QPS

确实，如果我们加的是一把全局锁，比如锁住整个商品 ID，那确实会让所有请求排队，性能直接退化成串行，QPS 肯定上不去。

但是，在我的设计里，我通过降低锁颗粒度完全规避了这个问题。

具体的逻辑是这样的：当高并发流量进来时，我的第一步并不是去抢锁，而是先利用 Redis 的 `DECR` 原子操作进行预分配。因为 `DECR` 是极快的，它会给每一个进来的请求瞬间分配一个唯一的库存索引，比如请求 A 拿到的是索引 99，请求 B 拿到的是索引 98。

拿到这个索引后，我再使用 `SETNX` 去加锁。注意，这时候请求 A 锁的是 `key_99`，请求 B 锁的是 `key_98`。在 Redis 看来，这完全是两个互不相关的 Key，它们之间没有任何竞争关系，是完全并行的。

所以，这套方案实际上是将高并发的资源争抢转化为了高并发的资源分配。系统的 QPS 瓶颈不再是锁竞争，而是单纯的 Redis 网络 I/O 吞吐量，而 Redis 单机轻松抗住几万 QPS 是完全没问题的。至于为什么要加 `SETNX`，那只是为了在极端异常情况下（比如服务宕机）能有一个可追溯的兜底，确保每一份库存的归属是唯一的，但这绝不会阻碍正常流量的并发执行。

总结一下

- **场景：** 缓存（Guava+Redis）、库存扣减、分布式锁（滑块锁）、Session/限流。
- **抽奖算法：** 为什么要用 Redis Map？（因为 $O(1)$ 查找，空间换时间）。
- **库存扣减：** 讲一遍Decr + SetNx的滑块锁逻辑。****
- **一致性：** Redis 扣减后 -> 延迟队列 -> Job 批量写库 -> 临界值（Stock=0）强制同步。
- **容灾与高可用：**
 - Redis 挂了怎么办？（Guava 兜底读，库存降级兜底写）。
 - 热点 Key 怎么办？（逻辑分片）。

nanobot

1. runtime

Runtime

`runtime` 说白了，就是 nanobot 活着并正在工作的那一层系统。

它负责管理

- 当前收到了什么消息
- 这条消息属于哪个 session
- 该把哪些上下文交给 agent
- agent 产生的结果怎么发回去
- 定时任务和主动唤醒怎么接进来

nanobot的runtime设计可以理解成一句话：

把输入、处理、输出和状态，收拢到一条统一链路里

1.1. nanobot 的 runtime 是怎么设计的

nanobot 启动时，会把几个核心对象一起拉起来：

- `MessageBus`：统一收消息、发消息
- `AgentLoop`：真正处理消息
- `SessionManager`：保存和恢复会话
- `ChannelManager`：管理 Telegram / Discord / WhatsApp 这些渠道
- `CronService`：定时任务
- `HeartbeatService`：定期唤醒，检查有没有事要做

这套东西可以抽象成三层：

1.1.1. 1. 渠道层

负责和外部世界通信。

聊天平台收到消息后，不直接调用模型，而是先转成统一格式，再交给 runtime。
回复用户时，也不是 agent 直接发，而是交回渠道层去发送。

1.1.2. 2. 总线层

负责在模块之间传递消息。

`MessageBus` 很朴素，就是两个队列：

- inbound：外部消息进来
- outbound：系统回复出去

总线就做搬运统一格式的消息，跟我们学的os里面的总线是一样的

1.1.3. 3. Agent 层

这里是负责真正理解消息、调用模型、执行工具的地方。

这一层的核心是 `AgentLoop`。

它从 bus 里取消息，找到对应 session，拼上下文，调模型，必要时执行工具，最后把结果再塞回 outbound 队列。

1.2. 一条消息怎么在 runtime 里流动

比如你在 Telegram 发一句话给 nanobot，大致会经过这几步：

1. 渠道适配器收到消息
2. 把消息转成统一的 `InboundMessage`
3. 丢进 `MessageBus.inbound`
4. `AgentLoop.run()` 从 inbound 队列取消息
5. 找到对应 session，恢复历史
6. `ContextBuilder` 拼上下文
7. 调模型
8. 如果模型要调用工具，就执行工具，再把结果喂回模型
9. 拿到最终回答后，写回 session
10. 把回复放进 `MessageBus.outbound`
11. `ChannelManager` 从 outbound 里取回复，发回对应平台

压缩成一句话就是：

聊天平台 -> MessageBus -> AgentLoop -> MessageBus -> 聊天平台

1.3. runtime 里最重要的设计

1.3.1. 1. 所有输入输出都走同一条链路

nanobot 没有让渠道直接调用 agent，也没有让 agent 直接操作聊天平台。

所有消息都先进入 bus

所有结果也都先进入 bus

这么设计的好处在于

- 渠道和 agent 解耦
- 消息流向清楚
- 调试时容易看清问题出在哪一层

1.3.2. 2. 被动消息和主动任务复用同一套 runtime

`CronService` 和 `HeartbeatService` 不是单独再造一套执行逻辑。

它们最后还是把任务送回 agent 主链路里处理，也就是说：

- 用户发来的消息，走这条 runtime
- 定时任务触发的消息，也走这条 runtime
- heartbeat 唤醒后的任务，还是走这条 runtime

这个设计很好，因为系统只有一套怎么处理任务的标准路径，简单的就是好的，好维护，也好读

1.3.3. 3. runtime 先解决怎么跑，agent 再解决怎么想

一个优秀的工程项目应该是解耦的，很多人一说 agent，就只会去做 prompt、tool call、memory。但 nanobot 的顺序是反过来的

它先把消息怎么进来、怎么出去、怎么归属 session、怎么避免串线，这些 runtime 问题处理好；再让模型进入真正的推理闭环

1.4. runtime 总结

AgentLoop 解决的是这条消息怎么被模型处理，

runtime 解决的就是这条消息怎么安全地进入系统、经过处理、再稳定地离开系统。

nanobot 的 runtime 很完整，它提供了一条统一的链路

- 渠道负责收发
- bus 负责转运
- agent 负责处理
- session 负责保存状态
- cron 和 heartbeat 负责主动触发

先把系统跑稳，再让 agent 变聪明

2. Agent loop

nanobot 的 **agent loop**，本质上是在把一条用户消息安全地变成一次完整回复。它分成两层：外层负责消息调度，内层负责模型推理。先把并发、session 和输出链路处理稳定，再让模型进入 **LLM** -> tool calls -> tool results -> LLM 的闭环。

一条消息的大致路径是：

收到消息 -> 找到 session -> 拼上下文 -> 调模型 -> 执行工具 -> 再调模型 -> 发回结果 -> 保存历史 -> 必要时压缩记忆

2.1. 外层：消息调度

外层循环持续从消息总线取消息，然后按 session 分发处理。这里最重要的设计只有一条：

同一个 session 串行，不同 session 并发。

只要同一会话里的两条消息交错进入推理流程，历史、工具结果和最终回复就会串线

2.2. 上下文怎么拼

消息进入 session 后，nanobot 不会立刻调用模型，而是先恢复这次推理需要的上下文。

上下文主要由五部分组成：

1. system prompt
2. 最近未归档的会话历史

3. 长期记忆 `MEMORY.md`

4. skills 和运行时信息

5. 当前用户消息

这里有一个很好的小设计：runtime metadata 会被注入上下文，但它被明确当作 metadata，而不是新的指令来源。这样模型能知道“现在在哪个 channel、什么时间、什么会话”，又不会把这些运行信息误当成用户要求。

2.3. 内层：推理闭环

上下文准备好之后，nanobot 才进入真正的 agent loop。这个 loop 很标准：

1. 把当前 `messages` 和可用工具定义发给模型
2. 如果模型返回普通文本，就结束
3. 如果模型返回 `tool_calls`，先把这条 assistant tool call 写进消息历史
4. 并发执行所有工具
5. 把每个工具结果以 `role=tool` 追加回消息历史
6. 再次调用模型，继续下一轮判断

这里最值得讲的设计点是：

assistant 的 tool call 先入历史，tool result 后入历史。

这个顺序不能反。因为 tool result 必须依附在一条合法的 assistant tool call 后面；只有这样，下一轮恢复历史时，这段推理链才是完整的，不会出现孤立的工具结果。

另一个值得保留的细节是：如果同一轮有多个工具调用，nanobot 会并发执行。它默认相信模型的表达：既然模型把多个工具放在同一批调用里，就说明这些调用之间没有强依赖。即使某个工具失败，系统也不会立刻打断整轮推理，而是把错误结果回灌给模型，让模型自己决定是重试、绕过，还是降级回答。

用代码描述推理loop可以是这样的：

```
1 while iteration < max_iterations:
2     resp = llm(messages, tools)
3
4     if resp.has_tool_calls:
5         messages += assistant(tool_calls=resp.tool_calls)
6         results = await gather(execute(tool_calls))
7         messages += tool(results)
8         continue
9     else:
10        messages += assistant(resp.content)
11        break
12
```

2.4. 结果怎么发出去

普通回答可以一次性返回；流式回答会拆成连续的 delta；如果模型中途要调工具，当前流会先结束，等工具执行完再恢复新的流段。除此之外，nanobot 还可以发送 progress 和 tool hint，让用户在最终答案出来前，先知道系统正在做什么。

主要解决把agent目前正在想什么安全地暴露给外部渠道。

2.5. 历史怎么保存

每次处理完成后，nanobot 会把这一轮新增消息写回 session，并保存到 `sessions/*.jsonl`。

有一些实现上的细节

- runtime metadata 会被剥离
- 过大的工具结果会被截断
- 高体积内容会被替换成轻量占位符

目的只有一个：后续恢复历史时，既保留语义完整性，又避免上下文无限膨胀。

2.6. 记忆怎么压缩

会话一长，就不能把全部历史一直喂给模型。nanobot 的做法很简单：不用向量数据库，而是做文件化分层记忆。

- `sessions/*.jsonl` : 完整会话流水
- `memory/MEMORY.md` : 长期事实
- `memory/HISTORY.md` : 旧事件摘要
- `last_consolidated` : 归档游标

当上下文快超出 token 预算时, nanobot 会从旧消息里挑出一段, 在用户轮次边界上做 consolidation。重要事实写进 `MEMORY.md`, 事件摘要追加到 `HISTORY.md`, 然后把 `last_consolidated` 向前推进。原始消息不会删除, 只是默认不再直接回灌给模型。

这个设计的优点很明确: 简单、可读、稳定。它没有复杂召回链路, 但很容易理解, 也很容易维护。

2.7. 结束条件

一次 `agent loop` 正常结束, 通常只有三种情况:

1. 模型不再请求工具, 直接给出最终回答
2. 达到最大迭代次数, 系统停止继续循环
3. 结果已经通过 `message` 之类的工具主动发出, 本轮不再补发重复回复

2.8. 总结

nanobot 的 `agent loop` 不是单次模型调用, 而是一条完整运行链路。它先把消息调度做稳, 再进入模型与工具的推理闭环; 同一 session 串行, 不同 session 并发; 历史不靠删除, 而靠 `MEMORY.md + HISTORY.md + last_consolidated` 做分层压缩。

我先按你的要求试了 `perplexity-search` skill, 但本地缺 `LiteLLM` 和 `OPENROUTER_API_KEY`, 没法直接跑起来。所以我用官方文档做了同类核对, 补了几条当前通用的 prompt 设计原则: OpenAI 强调消息角色优先级和“固定前缀放前、动态内容放后”以便缓存; Anthropic 强调 role prompt、固定/变量分离、以及把不同内容块明确分隔开。[OpenAI Prompt Engineering](#) · [OpenAI Prompt Caching](#) · [Anthropic System Prompts](#) · [Anthropic Prompt Templates](#)

下面这版正文, 我按 `db-content` 的要求做了减法, 只保留最该讲清的主线和最值得分享的小设计。

3. Context / Prompt

对 agent 来说，`Context / Prompt` 不是一段随手拼出来的提示词，而是这一轮推理的输入结构。它决定模型这一轮能看到什么信息，哪些信息优先级更高，哪些只是运行时背景，哪些属于当前用户输入。

nanobot 对这件事的处理很朴素：把上下文拆成几层稳定的部分，然后按固定顺序组装。

3.1. nanobot 的 Context 由什么组成

一轮请求真正送给模型之前，nanobot 会把上下文拼成下面几层：

1. system prompt
2. 最近未归档的会话历史
3. 当前这一轮用户消息

但 system prompt 本身又不是单层，而是继续拆开的：

- identity 和 runtime 环境说明
- 工作区里的 bootstrap 文件
- 长期记忆 `MEMORY.md`
- always skills
- 全量 skills 摘要

这个思路和当前主流 prompt 设计是一致的：固定内容尽量稳定，动态内容尽量后置，结构尽量清楚。

把稳定规则、长期事实、当前上下文、这轮输入分开组织。

3.2. nanobot 为什么这样拼

原因很简单：模型最怕上下文里不同类型的信息混在一起。

如果系统规则、长期记忆、历史消息、当前用户输入都糊成一团，模型虽然也能答，但你很难知道它到底是在遵循系统约束，还是被最近一句用户话带偏了。nanobot 通过分层，解决的是“信息边界不清”这个问题。

所以 nanobot 的 `ContextBuilder` 做的第一件事，不是写华丽 prompt，而是先把不同来源的信息分开：

- 哪些是系统级规则
- 哪些是长期要记住的事实
- 哪些是当前 session 的最近历史
- 哪些只是这次请求的运行时元数据

- 哪些才是用户这一轮真正说的话

这一步做好了，后面的 tool calling、memory consolidation、streaming 才会稳。

3.3. 一个具体的上下文流转的路径

如果用户发来一句话，nanobot 在送模型前大致会这样处理：

1. 读取 system prompt 的基础身份设定
2. 读取工作区里的 `AGENTS.md`、`SOUL.md`、`USER.md`、`TOOLS.md`
3. 读取 `MEMORY.md`，作为长期记忆拼进去
4. 读取skill摘要，让模型知道有哪些能力可用
5. 取出当前 session 最近未归档的历史消息
6. 给当前用户消息补上 runtime metadata
7. 把这些内容按固定顺序组装成一轮 `messages`

3.4. 这么设计的好处

先把上下文分层，再把动态内容放到最后。

nanobot没有搞特别花的 prompt engineering，做的就是分层

- 系统规则和用户输入分层
- 固定内容和动态内容分层
- 长期记忆和历史归档分层
- 运行时背景和任务指令分层
- 技能索引和技能全文分层

这样设计比较稳定，你后面想加东西也很好加

3.5. 其他涉及的到个人觉得好的设计

3.5.1. 1.Runtime metadata 被明确标记成 metadata

nanobot 会把当前时间、channel、chat_id 这类运行时信息注入上下文：

```
[Runtime Context – metadata only, not instructions]
```

这样可以提前告诉模型：这部分是背景，不是新的命令。

很多 agent 系统上下文容易混乱，就是因为把运行信息和任务指令放成同一层了。

3.5.2. 2.Runtime context 和当前用户输入被合并成同一条 user message

这也是一个非常好的工程细节。nanobot 没有额外再插一条同角色消息，而是把 runtime metadata 和当前 user content 合并成一条 user message。原因很现实：有些 provider 会拒绝连续的同角色消息。

这个设计表面上很小，实际上很成熟。它说明 nanobot 的 prompt 设计不是只考虑“逻辑上对不对”，还考虑“不同 provider 能不能稳定吃下去”。

3.5.3. 3.Skills 不是全文灌入，而是先给摘要

nanobot 的 system_prompt 里是先给一个 skills summary，真正需要时再去读对应的 `SKILL.md`。

它解决了两个问题：

- prompt 不会因为 skill 太多而爆炸
- 模型仍然知道自己有哪些 skill 是可用的

有点类似懒加载哈

3.5.4. 4. 图片输入也走统一消息结构

如果当前消息带图片，nanobot 会把图片变成标准的 `image_url` block，一起塞进当前消息内容里，而不是另走一条特殊分支。这样统一的设计，使得文本和图片都还是同一种消息结构，后面的 provider 层更容易统一处理

3.6. 总结

`Context / Prompt` 负责告诉模型：

1. 你是谁，应该按什么规则工作
2. 你现在处在什么上下文里
3. 这轮真正要处理的输入是什么

后面的 agent loop、tool calling、memory 和 streaming，都是建立在这个基础上的。

4. Memory

4.1. 总体设计

痛点：如果你跟nanobot聊得很久，原始聊天记录会越来越长，模型每次都把全部历史再看一遍会越来越贵、越来越慢，最后还可能超出上下文窗口。

方案：nanobot 的做法不是永远记住全部原文，也不是建一个复杂向量数据库，而是把记忆拆成了 3 份不同职责的东西进行存储，它靠**本地文件**长期存储，而不是数据库

- sessions/*.jsonl：完整聊天流水账
- memory/MEMORY.md：长期要点，这部分会被直接放进每次请求的系统提示词里，所以模型始终看得到
- memory/HISTORY.md：旧对话的事件日志，类似归档的作用。它默认不整份喂给模型，只在需要时搜索

4.2. memory是怎么工作的

第一次初始化工作区时，nanobot 会创建 memory/MEMORY.md 和 memory/HISTORY.md，其中 MEMORY.md 还有一个模板

```

nanobot > templates > memory > ⚡ MEMORY.md > abc # Long-term Memory
1  # Long-term Memory
2
3  This file stores important information that should
   persist across sessions.
4
5  ## User Information
6
7  (Important facts about the user)
8
9  ## Preferences
10
11 (User preferences learned over time)
12
13 ## Project Context
14
15 (Information about ongoing projects)
16
17 ## Important Notes
18
19 (Things to remember)
20
21 ---
22
23 *This file is automatically updated by nanobot when
   important information should be remembered.*
24

```

每次收到新消息时，大致会做这几步：

1. 先看看当前上下文会不会太大
2. 如果太大，就从最早的旧消息开始，挑一整段出来做压缩
3. 它会单独发起一次 LLM 调用，让模型调用 `save_memory` 的工具，并返回两样东西：
 - a. `history_entry`：这一段旧对话的事件摘要
 - b. `memory_update`：完整更新后的长期记忆全文
4. nanobot 把 `history_entry` 追加到 `HISTORY.md`，把 `memory_update` 覆盖写回 `MEMORY.md`
5. 然后它不删原始消息，只是把 `last_consolidated` 往前移动，这样子这些老消息已经归档过了，以后默认不再直接塞给模型（`last_consolidated`有点游标的感觉，告诉agent我目前整理到了这里）

4.3. memory设计里对于旧消息的处理

前面我们说到，nanobot会维护一个`last_consolidated`

nanobot不会真的把你所有的消息给删了，而是放在`sessions/*.jsonl`里

每次发给模型的历史消息，并不是所有的内容，而是`last_consolidated`后还没有被归档的部分。

last_consolidated的实际含义是前面有多少条消息已经被整理进 memory 了，get_history() 取历史时，直接从 self.messages[self.last_consolidated:] 开始切

举个例子：

1. 你和 nanobot 聊了 20 轮。
2. 前 12 条消息太老了，系统把它们归档。
3. 归档后：
 - a. 这些旧内容的重要事实写进 MEMORY.md
 - b. 事件摘要写进 HISTORY.md
 - c. last_consolidated = 12
4. 下一次请求模型时，历史窗口就只从第 12 条之后开始取。

4.4. 什么时候触发整理memory

核心理念是按 token 预算触发。

MemoryConsolidator 会估算当前 prompt 大小，再和上下文窗口上限比较；如果快超了，就开始归档，直到降到更安全的区间

有两个核心的规则：

1. 它会预留 completion token 和安全缓冲，不把上下文挤到极限

```

async def maybe_consolidate_by_tokens(self, session: Session) -> None:
    """Loop: archive old messages until prompt fits within safe budget.

    The budget reserves space for completion tokens and a safety buffer
    so the LLM request never exceeds the context window.
    """
    if not session.messages or self.context_window_tokens <= 0:
        return

    lock = self.get_lock(session.key)
    async with lock:
        budget = self.context_window_tokens - self.max_completion_tokens - self._SAFETY_BUFFER
        target = budget // 2
        estimated, source = self.estimate_session_prompt_tokens(session)
        if estimated <= 0:
            return
        if estimated < budget:
            logger.debug(
                "Token consolidation idle {}: {}/{} via {}",
                session.key,
                estimated,
                self.context_window_tokens,
                source,
            )
            return

```

2. 它只在用户轮次边界切块，避免把一组对话拦腰砍断

```

def pick_consolidation_boundary(
    self,
    session: Session,
    tokens_to_remove: int,
) -> tuple[int, int] | None:
    """Pick a user-turn boundary that removes enough old prompt tokens."""
    start = session.last_consolidated
    if start >= len(session.messages) or tokens_to_remove <= 0:
        return None

    removed_tokens = 0
    last_boundary: tuple[int, int] | None = None
    for idx in range(start, len(session.messages)):
        message = session.messages[idx]
        if idx > start and message.get("role") == "user":
            last_boundary = (idx, removed_tokens)
            if removed_tokens >= tokens_to_remove:
                return last_boundary
        removed_tokens += estimate_message_tokens(message)

    return last_boundary

```

4.5. 例子解析nanobot的memory设计

一开始在运行区，会默认创建模版MEMORY.md


```
1 ▾ # Long-term Memory
2
3   This file stores important information that should persist across sessions.
4
5 ▾ ## User Information
6
7   (Important facts about the user)
8
9 ▾ ## Preferences
10
11  (User preferences learned over time)
12
13 ▾ ## Project Context
14
15  (Information about ongoing projects)
16
17 ▾ ## Important Notes
18
19  (Things to remember)
20
21  ---
22
23  *This file is automatically updated by nanobot when important information
24  should be remembered.*
```

实际运行一段时间后，它可能会变成这样

```

1 ▾ # Long-term Memory
2
3 ▾ ## User Information
4 ▾ - Name: Martin
5   - Works mainly on nanobot and agent systems
6
7 ▾ ## Preferences
8 ▾ - Prefer Chinese explanations
9   - Prefer implementation details over marketing descriptions
10  - Likes concise but concrete examples
11
12 ▾ ## Project Context
13 ▾ - nanobot uses file-based two-layer memory
14   - Long-term memory is stored in memory/MEMORY.md
15   - Historical summaries are appended to memory/HISTORY.md
16
17 ▾ ## Important Notes
18   - User is currently studying how nanobot memory works internally
19

```

同时前面我们也讲了，如果上下文太长了nanobot是会触发压缩到HISTORY.md的

HISTORY.md 没有复杂结构，本质上是 append-only 日志文件。每次归档时，追加一条新记录，条目之间空一行

理想格式是模型通过 `save_memory` 返回一个字符串，要求开头带时间戳

```

1 ▾ [2026-03-26 14:30] User asked how nanobot memory works. Explained that memo
   ry is split into MEMORY.md for long-term facts and HISTORY.md for searchabl
   e archived events. Discussed that sessions stay in JSONL and old chunks ar
   e consolidated when token budget is exceeded.

```

多条累积后会像这样：

- 1 ▾ [\[2026-03-20 10:12\]](#) User discussed Telegram integration and decided to keep reply context minimal.
- 2
- 3 ▾ [\[2026-03-22 18:45\]](#) User asked about session persistence and restart behavior. Explained that sessions are stored as JSONL files and reloaded from disk.
- 4
- 5 ▾ [\[2026-03-26 14:30\]](#) User asked how nanobot memory works. Explained the two-layer design, token-triggered consolidation, and persistent storage files.

所以到这里你能直观地感受到一个简单的文件系统但是覆盖了多层记忆的设计理念，它真的足够简单

nano is all u need

模型记住了这些内容还不够，还需要有想起的能力，通过system_prompt和一些小技巧，nanobot实现了这一点

- MEMORY.md：每次构建 system prompt 时直接读出来，拼进 # Memory 段里，所以模型天然会看到。context.py (line 36)
- HISTORY.md：默认不自动注入 prompt，只是留作可搜索档案。
nanobot/skills/memory/SKILL.md
- sessions/*.jsonl：最近未归档的消息会继续作为对话历史直接喂给模型。manager.py (line 69)

跟我们上面讲的三层记忆对应上了：

- 眼前正在聊的：session 最近消息
- 必须长期记住的：MEMORY.md
- 更久远的历史细节：HISTORY.md

上下文压缩触发的细节是，当上下文太长了，nanobot会把一段旧消息格式化后发给模型，并强制模型调用 save_memory 工具，返回：

- history_entry
- memory_update

4.6. 模型真正记住了什么

1. 真正的长期记忆：

- a. MEMORY.md 里的内容，MEMORY每次都会被放进system_prompt里

```

class ContextBuilder:
    """Builds the context (system prompt + messages) for the agent."""

    BOOTSTRAP_FILES = ["AGENTS.md", "SOUL.md", "USER.md", "TOOLS.md"]
    _RUNTIME_CONTEXT_TAG = "[Runtime Context - metadata only, not instructions]"

    def __init__(self, workspace: Path, timezone: str | None = None):
        self.workspace = workspace
        self.timezone = timezone
        self.memory = MemoryStore(workspace)
        self.skills = SkillsLoader(workspace)

    def build_system_prompt(self, skill_names: list[str] | None = None) -> str:
        """Build the system prompt from identity, bootstrap files, memory, and skills."""
        parts = [self._get_identity()]

        bootstrap = self._load_bootstrap_files()
        if bootstrap:
            parts.append(bootstrap)

        memory = self.memory.get_memory_context()
        if memory:
            parts.append(f"# Memory\n\n{memory}")

        always_skills = self.skills.get_always_skills()
        if always_skills:
            always_content = self.skills.load_skills_for_context(always_skills)
            if always_content:
                parts.append(f"# Active Skills\n\n{always_content}")

        skills_summary = self.skills.build_skills_summary()
        if skills_summary:
            parts.append(f"# Skills

```

b.

2. 可检索的内容:

- a. 区别记忆与检索, HISTORY.md 放的是可检索的内容
- b. 适合放发生过的事件
- c. 决策过程某次调试、某次讨论、某天的结论

4.7. memroy设计总结

nanobot这套memory设计, 没有用向量数据库, 也没有语义检索系统, 可以说确实符合它的sologn——nano

这样设计的好处是

- 简单, 直接用文件, 不依赖外部数据库
- 可读, MEMORY.md 和 HISTORY.md 你自己都能打开看, 自己也可以修改
- 稳定, 不需要 embedding、向量索引、召回排序这些更复杂的链路

- 便宜，减少每轮 prompt 长度

trade-off呢？

- nanobot的memory设计不是语义检索系统，所以不会自动从历史里最相关地召回第 237 次对话，质量很大取决于当前history和memory的质量，也就是说长期记忆质量很依赖整理那次 LLM 输出是否靠谱。
- HISTORY.md 里的内容需要搜索，不会默认全部回灌给模型。
- 所以代码里做了失败兜底：如果记忆整理连续失败 3 次，就直接把原始消息 raw dump 到 HISTORY.md，至少不丢记录。

5. skill

对于agent设计中的skill来讲，有几个问题需要考虑

1. skill怎么进入上下文：
2. 哪些规则应该常驻，哪些skill先只给摘要，哪些说明等模型判断需要时再展开。

在 nanobot 里，skill 本质上是一份外置的能力说明。它通常是一个目录，最少包含一个 `SKILL.md`，必要时再带少量脚本、参考资料或资源文件。工具负责执行动作，skill 负责补上动作前面的经验：什么时候该用、怎么用、依赖什么、边界在哪里。

5.1. 为什么需要skill

nanobot 的 system prompt 里本来就有固定内容，例如身份、工作区约束、bootstrap 文件和 memory。

再把所有 skill 的正文一起放进去，问题不只是 token 成本更高，更重要的是当前任务会被无关说明稀释。

问天气时，不需要同时背着 GitHub、tmux、cron 和自定义工作流的完整用法。

所以，Skills 这一层解决的是上下文分层。稳定的规则保留在 prompt 里，变化更快、只在特定任务里有用的能力说明放到外部文件里。这样做也符合当前常见的 context engineering 做法：把稳定前缀和按需信息分开，避免每一轮都重灌整套说明。

5.2. nanobot的实现思路

nanobot 会先扫描工作区里的 `skills/`，再扫描内置的 `nanobot/skills/`；如果名字重复，工作区版本优先。这个顺序很重要，因为它决定了本地定制可以直接覆盖框架默认行为。

进入 prompt 时，skills 被分成两层。第一层是标记了 `always: true` 的 skill，会直接进入 `# Active Skills`。当前最典型的例子是 `memory`，因为它属于 agent 每轮都要知道的运行规则。第二层是所有 skill 的摘要，由 `build_skills_summary()` 生成，只保留名称、描述、位置、当前是否可用，以及缺失的依赖。模型平时看到的是这份摘要；只有当某个 skill 真和当前任务相关时，才再去读取对应的 `SKILL.md`。

nanobot 先给模型一份 skill 目录。模型先判断“这项能力值不值得展开”，再决定要不要读正文。这样，skill 的入口信息很短，正文就可以写得更实，不必为了节省上下文把所有说明压成几句含混的话。

5.3. 一些小设计

1. 不可用的 skill 不会被静默隐藏。摘要里会明确写出 `available="false"`，并附上缺失条件，例如少了某个 CLI 或环境变量。这样模型知道系统里本来有这项能力，只是当前环境还不满足；这比完全不可见更实用。
2. 同名覆盖采用最朴素的规则：工作区优先。没有额外注册流程，也没有复杂优先级表。
3. skill 的入口信息和正文被明确分开。对 nanobot 来说，前者主要放在 frontmatter 里，例如 `name`、`description`、`always`，以及 `metadata` 里的 `requires`；后者留在 `SKILL.md` 正文和附带资源里。这让模型先用很少的信息完成选择，再决定是否展开细节。
4. skill 目录本身也有边界。`quick_validate.py` 会限制根目录结构，只允许 `SKILL.md`、`scripts/`、`references/`、`assets/`，还会校验 skill 名称、目录名称和占位描述。这种约束看起来保守，但它换来的是可读性和可维护性。

5.4. 总结

nanobot 在设计 skill 能力的时候，比较注重结构化。

把 skill 分成常驻注入和按需展开两层，而不是把所有正文一次性塞进 prompt。工作区覆盖、可用性标注和目录校验，让 skill 扩展保持可读、可控、可维护。

6. provider

这块的设计就是解决一个问题：换模型而不用重写 agent 这一章讲 nanobot 的 Providers。

不同厂商的 API、消息格式、工具调用协议和流式返回都不一样，但 agent loop 不应该跟着一起变复杂。

Provider 负责把外部模型服务的差异收敛成 nanobot 内部可消费的一套统一接口。

对上层来说，一次调用的结果无非两种：模型给出文本，或者模型请求调用工具。至于底层到底是 Anthropic 的消息块、OpenAI 兼容接口、Azure 的部署名，还是带额外字段的工具调用，这些都应该留在 provider 层里解决。

nanobot 需要这一层，首先是因为 agent loop 的职责应该尽量单纯。负责拿上下文、调用模型、执行工具、回灌结果、继续下一轮推理。如果模型接入方式也混在这一层里，那么每增加一个 provider，上层就要知道新的消息格式、新的错误类型、新的工具调用结构，系统会很快失去边界。provider 层的意义，就是让 agent loop 只依赖稳定的内部结构，而不是直接依赖某一家厂商的协议。

nanobot 的做法不是去强行统一所有 provider 的请求格式，而是统一内部最关键的返回结构。LLMResponse 承接一轮模型调用的结果，ToolCallRequest 承接工具调用请求。前者统一了文本、finish reason、usage、reasoning 内容和 thinking blocks，后者统一了工具名、参数和工具调用 ID。这样一来，agent loop 看到的始终是同一种结果对象，而不需要知道底层 SDK 返回的是对象、字典，还是流式增量片段。

可以把这层抽象理解成这样：

```
1  @dataclass
2  class ToolCallRequest:
3      id: str
4      name: str
5      arguments: dict
6
7  @dataclass
8  class LLMResponse:
9      content: str | None
10     tool_calls: list[ToolCallRequest]
11     finish_reason: str
```

有了这层统一结构，agent loop 的写法就可以一直保持稳定：

```
1  resp = provider.chat_with_retry(messages, tools, model)
2
3  if resp.tool_calls:
4      run_tools(resp.tool_calls)
5  else:
6      return resp.content
```

外面怎么变，上面这段主逻辑都不用跟着变。

由注册表里的 `ProviderSpec` 把 provider 的核心信息集中放在一处：名称、匹配关键词、后端类型、默认 API base、是否是 gateway、是否是本地模型、是否支持 prompt caching、是否需要 OAuth。配置层再根据模型名、provider 前缀、API key、API base 和回退规则，把当前模型路由到正确的 provider 实现。

这个选择过程大致可以压成下面这样：

```
1 provider_name = config.get_provider_name(model)
2 spec = find_by_name(provider_name)
3
4 if spec.backend == "anthropic":
5     provider = AnthropicProvider(...)
6 elif spec.backend == "azure_openai":
7     provider = AzureOpenAIProvider(...)
8 elif spec.backend == "openai_codex":
9     provider = OpenAICodexProvider(...)
10 else:
11     provider = OpenAICompatProvider(...)
```

实现层面，nanobot 当前的 provider 结构很清楚。大部分 OpenAI 兼容接口都走 openai，Claude 走单独的，Azure OpenAI 和 OpenAI Codex 再各自保留单独实现。因为 Anthropic 就是一个典型例子：它有自己的消息块结构、tool result 组织方式、thinking block 和 prompt caching 机制，继续硬塞进 OpenAI 兼容接口只会让兼容层越来越脏。

Anthropic provider 做的事情，本质上是协议翻译。nanobot 内部沿用一套更接近 OpenAI 风格的消息结构，但 Claude 原生接口要求另一种块状结构，所以 provider 会在发请求前先转换：

```
1 system, anthropic_msgs = convert_messages(openai_style_messages)
2 anthropic_tools = convert_tools(tools)
3
4 response = anthropic_client.messages.create(
5     system=system,
6     messages=anthropic_msgs,
7     tools=anthropic_tools,
8 )
```

具体的细节就不过多展开了，如果有需要实现的时候可以对照着代码问下 AI，都是兼容上的问题，偏工程化

7. Subagent

问题：当主对话里出现耗时、可独立完成任务时，系统怎么把它放到后台去跑，又在跑完之后自然地把结果接回当前会话

nanobot的方案：用subagent，subagent 是一个临时的后台执行单元。主 agent 通过工具把任务交出去，后台单独完成，完成后再把结果送回主 agent，由主 agent 继续对用户说话。整个过程保留了单一会话的外观，但把执行和回复拆开了。

7.1. 为什么需要subagent

如果所有任务都堵在主对话这条链路里，用户一旦发起搜索、读写文件、执行命令这类耗时操作，就只能原地等待。系统表面上简单，实际体验却很差，因为一次回复被迫承担了持续执”的责任。

但 nanobot 也没有把这个问题升级成一套复杂的多 agent 编排。它没有让多个角色互相讨论，也没有给每个后台任务独立的人设和对话权限。因为我们只需要把一部分工作挪到后台，而不是重新发明一套社会化 agent 系统。

7.2. nanobot的实现思路

可以，后面这一段最该补的是 **Subagent**。这一章不用讲成多 agent 理论，直接围绕 nanobot 现有实现写就够了。

7.3. Subagent

如果所有任务都堵在主对话链路里，用户一旦发起搜索、读写文件、执行命令这类耗时操作，就只能原地等待。系统表面上简单，实际体验很差，因为一次回复被迫同时承担两件事：一边保持当前会话，一边长时间执行后台工作。

nanobot在这块的设计没有做角色协作的复杂编排，没有给后台任务独立人格，也没有让多个 agent 互相讨论。它对 subagent 的设计非常克制：只解决后台执行，不扩大成新的系统中心。

7.4. nanobot 的实现思路

主 agent 侧暴露的是一个普通工具：

```
1 spawn(task="去后台检查这个站点的首页并总结结果", label="check-homepage")
```

一旦模型调用 `spawn`，`SpawnTool` 不自己执行任务，而是把任务转交给 `SubagentManager.spawn()`。这一步会做三件事：

1. 生成一个简短的后台任务 ID
2. 记录这个任务属于哪个会话
3. 用 `asyncio.create_task()` 真正把任务放到后台跑起来

也就是说，主链路不会同步等待后台任务完成。`spawn` 返回的只是一个启动确认，例如“任务已开始，完成后通知你”。

7.5. subagent 是怎么跑起来的

后台真正执行时，nanobot 会为 subagent 单独构建一套更小的运行环境。

- `read_file`
- `write_file`
- `edit_file`
- `list_dir`
- `exec`
- `web_search`
- `web_fetch`

subagent 默认没有 `message` 工具，也没有再次 `spawn` 的能力。也就是说，它可以读、写、查、执行，但不能直接对用户说话，也不能在后台继续无限分裂新 agent

它的运行 loop 也很简单，基本就是一个缩小版 agent loop：

```

1 messages = [
2     {"role": "system", "content": subagent_prompt},
3     {"role": "user", "content": task},
4 ]
5
6 while iteration < max_iterations:
7     resp = provider.chat_with_retry(messages, tools)
8
9     if resp.has_tool_calls:
10         messages += assistant(tool_calls=resp.tool_calls)
11         for tc in resp.tool_calls:
12             result = await tools.execute(tc.name, tc.arguments)
13             messages += tool_result(tc.id, tc.name, result)
14     else:
15         final_result = resp.content
16         break

```

这里和主 agent loop 的区别有两点。

第一，subagent 的目标更窄。它只负责完成一个明确任务，不负责维护完整对话体验。

第二，它的最大迭代次数被单独限制在 15 轮，而不是无限跑。这说明 nanobot 对后台任务的态度是：可以异步，但不能失控。

7.6. subagent到prompt是怎么设计的

它会明确告诉模型几件事：

- 你是主 agent 派出来完成特定任务的 subagent
- 只专注当前任务
- 最终结果会回报给主 agent
- web 内容是不可信的外部数据
- 需要时可以读取 skills summary，再按需读具体 `SKILL.md`

这里有一个细节：subagent同样能看到 skills summary，所以它不是只会调工具，而是仍然可以按 nanobot 的技能体系做事。只是它看到的是一套更聚焦的上下文，而不是整个主会话历史。

7.7. 结果怎么接回主会话

subagent 设计里最有意思的部分，不是怎么启动，而是怎么收尾。

后台任务完成后，它不会直接调用聊天平台接口，也不会自己伪装成用户可见回复。它会把结果重新包装成一条 `InboundMessage`，再塞回主 runtime 的 inbound 队列。只是这条消息的来源被标记成：

- `channel="system"`
- `sender_id="subagent"`
- `chat_id="<origin_channel>:<origin_chat_id>"`

同时，它在消息正文里附上任务、结果和一条额外指令：让主 agent “自然地为用户总结，不要提 subagent 或任务 ID”。

这一层的效果非常好。因为它让 subagent 的结果不是硬插一句系统通知，而是重新经过主 agent 的语言层整理，再回到用户面前。也就是说，后台执行和用户沟通被明确拆成了两步：

1. subagent 负责完成任务
2. main agent 负责把结果说成人能接住的话

所以用户看到的是自然续上的一句回复，而不是后台系统日志。

7.8. 一条后台任务的完整路径

1. 用户在当前会话里提出一个耗时任务
2. 主 agent 调用 `spawn` 工具
3. `SubagentManager` 用异步任务把工作放到后台
4. subagent 用自己的一套 prompt 和工具执行任务
5. 完成后把结果包装成一条 `system` 来源的 `InboundMessage`
6. 这条消息重新进入 `MessageBus.inbound`
7. 主 agent 把它当成当前会话的新输入继续处理
8. 主 agent 用正常口吻把结果告诉用户

主 agent 派活 -> subagent 后台执行 -> 结果回灌 bus -> 主 agent 自然转述

7.8.1. 一些好的设计

1. subagent 不直接回复用户。
它只把结果送回主链路，最终的话仍然由主 agent 来说。这保证了用户体验是连续的，而不是突然冒出另一种系统口吻。
2. subagent 有独立工具集，但工具集被故意缩小。
没有 `message`，没有 `spawn`。这样它能做事，但不至于脱离主链路自成系统。

3. subagent 和主 agent 复用同一个 provider。

它没有为了后台任务再造一套模型调用逻辑，而是沿用同样的 provider 抽象。这样前后台的模型行为更一致，维护成本也更低。

4. nanobot 还维护了按 session 的后台任务索引。

这意味着当前会话可以取消自己拉起的 subagent，而不用碰别的会话的后台任务。后台执行虽然异步，但归属关系仍然清楚。

7.9. 结束条件和边界

subagent 的结束条件也很朴素：

1. 模型不再请求工具，给出最终结果
2. 达到最大迭代次数，停止继续执行
3. 运行中抛出异常，按失败结果回灌主会话

7.9.1. 总结

1. subagent 不是新角色，而是主 agent 派出的临时后台执行单元。
2. 它做完事不直接回复用户，而是把结果重新送回主链路，再由主 agent 转述。
3. 用最小复杂度给 nanobot 增加后台执行能力。

Java后端开发求职分享-我的实习_秋招复盘

写在前面：秋招就像滚雪球，早点准备，能把自身的优势发挥出来，学历一般的朋友不要气馁，早点准备找到份好实习秋招照样通杀，学历好的朋友也别坐以待毙，没有实习在现在的就业环境寸步难行

个人bg：本2硕9 科班

求职方向/base：后端开发 深圳/成都

求职完整时间线：

24年底断断续续学完java基础

25年1月正式主要精力用于求职写项目

25年3月打磨好简历第一次投递

一个月内拿下美团/字节/快手offer

某互联网公司第一段实习4个月

25年7月入职第二段互联网实习

利用下班时间准备秋招/实习转正

25年11月结束秋招投递/面试

上岸字节/美团/京东等互联网大厂

Java基础

核心：快速看，有个印象，不要被大而全的课程吓到了，很多东西其实根本不用看了解一下就好，哪怕漏了后续写项目的时候自己查漏补缺即可

因为本科有Java基础，这部分学习主要是找不会的地方，b站上视频很全，我当时看的是



【尚硅谷】7天搞定Java基础，
Java零基础极速入门

UP 尚硅谷

90.5万 · 2022年11月15日



这些类的视频大差不差，我建议选个少的，声音倍速快起来听着舒服的，

如果0基础转码：着重看下环境配置，基础语法

如果有一定基础：倍速看，主要看继承多态重载，接口，抽象，枚举，异常，ArrayList，LinkedList，HashMap，多线程

其他的有个大概印象就行

Spring全家桶

核心：学会怎么CRUD

学完了Java se，就可以看些框架，大多数公司都是用spring，我当时看的是



【狂神说Java】Spring5最新完整教程IDEA版通俗易懂

UP 遇见狂神说

255.5万 · 2019年10月13日



一键空降 · 共28集

0:01 1、Spring简介 ▾



【狂神说Java】SpringBoot最新教程IDEA版通俗易懂

UP 遇见狂神说

498.4万 · 2019年11月10日

这里面也会讲一下mybatis/redis等其它的内容，不懂的话可以网上搜或者gpt问

我是不建议学完javase就去单独学mysql/redis，大多数学校本科都会开数据库的课程，因此很多视频对一些没做过项目的同学来说没什么意思，没有正反馈，因此我redis和mysql都是查漏补缺，遇到不会的再去看

Mysql

mysql的学习主要有两部分

1. 能够应对面试可能的sql手撕：牛客上有很多sql题目，写个几十道并整理笔记就完全够用了
2. 能够应对面试的八股：mysql八股属于重中之重，基本每场都会考察，这部分建议直接看 小林coding 的mysql，自己做笔记，后续我也会把笔记整理出来分享

JVM

这一块是项目结束后开始学的，大概是在25年的2月中旬

学习的资源主要是java guide里的八股和狂神

先倍速看完了狂神上的内容



【狂神说Java】JVM快速入门篇

UP 遇见狂神说

74.8万 · 2019年11月23日

然后去刷java guide上的八股，遇到不会的就gpt补充，最终整理成自己的八股

可以重点看java内存区域，垃圾回收算法，垃圾回收器，类加载器

整体耗时大概接近一周，jvm有些内容还是需要好好思考的

JUC

这块整体面试下来占比不多，且大多数都是那几个问题，我面试准备的时候是还去看了视频，现在回过头看不推荐

直接去看java guide就好了

重点看进程线程关系，volatile关键字，乐观锁悲观锁，cas，threadlocal，aqs

整体集中学习的时间在3天左右

消息队列

这个根据你项目里面的选型，如果是rocket mq就看对应的文档学习，如果是kafka也同理

对于实习来说，你在面试的时候一般是上面有什么（rabbitmq/kafka），面试官就问你什么

这块的内容主要是从项目学

之后还要重点看下消息队列里的场景八股

比如消息队列怎么用来异步处理，削峰限流，降低系统耦合，实现分布式事务

Redis

这块的主要学习是写黑马点评这个项目



黑马程序员Redis入门到实战教程，深度透析redis底层原理+...

UP 黑马程序员

485.5万 · 2022年2月28日



内容很多，但作为一个“烂大街”的项目，我当时的思路是不在“接受课程内容”上花过多时间，比如某个细节的实现，某行代码怎么写，我主要是面向面试学习，根据课程大纲让gpt帮我生成可能的重点模块，然后去看每个模块是怎么实现的，redis在里面起了什么作用，我能不能改

所以我花时间较多的是自己整理笔记，有点像写技术设计文档，把框架（比如ppt上的一些关键图），核心技术点（比如多级缓存，秒杀优惠券演变的过程等）这些一一整理，做到能够口述方案，模拟面试的效果，那基本这一块内容就算学扎实了

因为这一块比较重要，我还是给大家重复一遍：

1. 把完整的项目导入
2. 对着课程视频看，不要想着怎么一行一行代码写完整个项目，抽象成每个模块怎么实现，转换成一个问题带着思考（比如整个项目秒杀模块优惠券是怎么演变的）
3. 整理成自己的笔记，打磨面试话术（库存不超卖的实现，高并发下高性能怎么做...）

项目

我觉得可以算是准备实习最重要的环节

通过项目你可以减少面试前的恐惧感（我还是做了点文具项目的），面试中的卡顿可能（我已经有了项目准备了话术，来问我吧），同时也是减少学习成本快速掌握后端核心技术栈的方式

篇幅的原因，在这里我给大家分享一下怎么在三个月内快速地完成一个找实习用的项目

项目的选择上：点评/各大知识星球项目都可以，核心就是找资源多的

准备工作：环境搭建，搭好项目其实等于做一半

面向面试学习：找到关于这个项目的优秀简历描述和面经，看看大家都是怎么总结的，会遇到什么面试问题

调整学习时的输入/输出占比：看项目视频的时候看更多侧重在项目为什么这么做，技术选型，整体架构上，对于每一行实现细节不需要钻牛角尖，更重要的是自己总结各个模块可能遇到的面试问题并整理话术，再提炼成为简历上的文字

我大概花了一个半月做完了点评并写好简历，然后发现了一个不错的知识星球，再花了20天左右做了其中一个项目的前半部分，写到了简历

关于项目更多的内容后续会更新帖子专门讲

算法

现在的面试来说，如果手撕不出来大概率是挂的

因此也要注重算法的训练

这里分享一下我怎么学的：

1. 实习准备只刷hot100上的题，基本能做到看到就秒，在刷之前我让gpt帮我整理了hot100大概的分类，每个分类按顺序刷
2. 秋招的话我还看了代码随想录，分模块的刷可以构建体系，上面的内容还是比较详细的

其他的，我还会在面试具体公司的时候刷刷面经，有一次还真碰到面经里的算法题hhh

每天保持一两道的手感，在投递前一两周集中刷

27找实习的一个合理时间线

已经有点java基础的同学，目标是三月四月投递

1. 一月抓紧做项目，开始每天一道hot100保持手感，多留心求职信息（招聘信息，面经帖子）
2. 二月重点开始把项目中遇到的后端核心技术栈，以三天为一个时间段，快速地学一下，主要是面向面试学习，即项目中用到了xx，他是干什么的，他在我的项目里做什么，他的八股我能讲几个
3. 三月初打磨自己的简历，主要是整理项目话术
4. 三月底四月初之前：集中血八股，复习项目，这个时候每天学习的时间起码要到5h，也是最累的时间段，坚持💪
5. 投递直至实习入职：每次面试录音并复盘，完善八股和项目上的欠缺

没有java基础的同学

在上面的时间节点上，目标是5/6月投递

先花一个月入门java 补全自己的一些科班欠缺，比如git mysql这些

之后直接做项目，在做项目的过程中学习，不要陷入大而全的技术学习上

尽量赶上4/5/6月的暑期实习投递黄金期

点评_外卖之类的“烂大街”项目还值得做吗

点评/外卖之类的“烂大街”项目还值得做吗

先说结论：值得

点评/外卖作为很好的入门项目，网上有大量的视频/笔记可以参考借鉴，节省我们准备的时间

但切记全部照抄，我们需要取项目中精华的内容写在简历上，**引导面试官考察**我们对项目的理解程度和学习能力，也就是准备自己对各模块的“话术”

很多没有面试经验的朋友，经常会问我，项目究竟会问什么？

其实不应该纠结问什么，而应该纠结，我写在简历上内容是什么，怎么引导面试官考察我准备好的话术

也就是把每个写在简历上的点，概括成 **背景 + 问题/难点 + 我的方案 + 达到的效果**

以点评的“秒杀性能优化”为例子：

- **背景**：在商户促销活动中，系统需应对瞬时爆发的高并发下单请求。初始方案采用同步下单，所有逻辑（校验、扣库存、写单）均在数据库事务中完成。
- **难点**：数据库行锁争用严重，QPS 遭遇物理瓶颈；在高并发下，传统的Java锁无法跨服务器，面临超卖风险及重复下单的幂等性挑战。
- **方案**：将校验逻辑前置并逻辑下沉。利用 **Redis + Lua** 脚本实现“预扣库存”与“下单资格校验”的原子化操作；校验通过后，直接将下单请求投递至 **Kafka 消息队列**，实现订单生成的异步化与流量削峰。
- **效果**：接口响应时间（RT）从百毫秒级降低至十毫秒级，彻底杜绝了超卖现象，系统吞吐量提升了数倍，成功保障了高并发下的服务稳定性。

这就是我为这个模块准备的话术，后续可以延申出一系列问题，这样面试的时候就跟面试官有话聊了，也能体现你对技术的思考

如果面试官问你这个项目是不是你自己的想法，作为实习求职/校招生，完全可以大大方方的承认是网上的项目，但自己的思考是1..... 2..... 3.....，我觉得对于XX模块，还有1/2/3处地方，可以优化，这样也能体现你的思考

一切的一切，都不是为了追求我把项目做完了，而是引导面试官往你熟悉的地方考察，展示你提前准备话术的深度，这样才能留下一个好的面评。

三个月内做完一个完整项目-面向面试学习

三个月内做完一个完整项目-面向面试学习

找实习最难的是什么？

不是背八股，也不是刷算法，而是简历上没有拿得出手的项目。

我也经历过这个阶段：看着网上的面经，觉得每个人都做过高并发、分布式

自己还在写CRUD，根本不敢投递。

但后来我发现，项目其实是我们这种学背景不占优势的同学

减少面试恐惧感、快速掌握核心技术栈的一条捷径。

但很多同学做项目有个误区：从头敲到尾，每一行代码都死磕。

结果往往是视频看了几百集，代码敲了一半就放弃了，简历上还是没东西。

接下来我会复盘我是如何在去年同样的时间点，在一个半月内做完点评项目

又花20天搞定大营销，最终三月份拿下大厂暑期Offer 的实战经验

三个月突击项目，到底该怎么做？

很多同学做项目有个最大的误区：把自己当成了一个纯粹的开发，而不是一个求职者。从头敲到尾，每一行代码都死磕，视频看了几百集，结果做了一半累得放弃了，简历上还是没东西。

我的核心方法论只有一条：面向面试做项目，面向简历写代码。

项目选型：不要追求标新立异

很多同学问我：“点评、外卖这些项目是不是烂大街了？面试官会不会看腻了？”

先说结论：烂大街的才是最好的。

作为校招/实习生，我们需要的是资料多、坑少、面经丰富的资源。

你想到的Bug，网上都有人踩过坑；面试官会问什么刁钻角度，牛客上都有人总结了。

推荐：点评、各大知识星球带文档的实战项目。别为了追求独特去搞什么偏门项目，那时候在这个项目里遇到坑，全网都没人能救你。

心法：输入输出倒置

做项目不是为了让你成为熟练的 CRUD 工程师，而是为了**积累面试素材**。

借着做项目的过程，去锻炼解决问题的思维。

① 学习时的另一个视角

看视频/文档时，千万别死盯着 `user.setName` 这种代码细节，毫无意义。

你要跳出来，站在**项目owner**的角度去审视：

- **Why**：为什么教程里这里选了 Redis？如果数据量小一点，用本地缓存 HashMap 行不行？
- **Trade-off**：为什么用 Redisson 实现分布式锁？它对比数据库悲观锁，牺牲了什么，又得到了什么？
- **Architecture**：一个请求进来，是怎么一步步流转的？哪里是性能瓶颈？

② 费曼学习法

每做一个核心模块（比如秒杀），立刻停下来，不要急着写下一行代码。

去搜这个模块的优秀简历描述和面试题

- **参考**：看看别人是怎么总结这个技术点的（背景-难点-方案）。
- **反思（最重要的一步）**：对比别人的描述和自己的代码。问自己几个“如果”——“如果我把这个MQ去掉会怎么样？”、“如果并发量再大10倍，我现在这个方案会不会挂？”
- **内化**：把别人的话术，结合你刚刚的思考，转化成你自己的语言写在简历上。

实战案例：如何输出自己话术

很多没有面试经验的朋友，经常问：“面试官会问什么？”

其实不应该纠结他问什么，而应该纠结：

我写在简历上的内容是什么？我怎么引导面试官往我准备好的坑里跳？

以点评的**秒杀模块**为例，不管面试官怎么问你

“介绍一下你的秒杀”、“你怎么解决高并发”

你都要把答案往你准备好的**【背景 + 难点 + 方案 + 效果】**这套框架上引。

以下是我整理的我做的一个项目DDD的话术

大家也可以参考我整理的方式，用我的格式进行整理

写在最后

大家发现了吗？这一套流程下来，你根本不是在被动回答问题，而是在讲一个故事。一个关于“系统遇到了什么问题 -> 我是怎么思考的 -> 我用了什么技术方案解决 -> 最终达到了什么效果”的故事。

如果面试官问你：“这个项目是你自己想的吗？”作为实习生，完全可以大大方方承认是参考了开源项目或者教程，但关键在于补充一句：“项目框架是参考的，但里面关于分布式锁的选型、异步消息的一致性处理，是我自己根据压测结果进行优化和思考的。”

这才是面试官想要听到的答案。

三个月时间，足够你从0到1把这个故事讲圆满。不要怕烂大街，把烂大街的项目讲出深度，才是你的本事。

祝大家都能拿到满意的 Offer！

小厂实习也可以是加分项

小厂实习也可以是加分项

在暑期实习申请中，中小厂的背书其实并非减分项

面试官评估候选人时，公司名气往往不是唯一尺度

一段含金量高的前期实习经历反而是绝佳的加分项

然而如果你在描述经历时仅流于表面

把实习内容整理成若干的CRUD

很难激起面试官的提问兴趣

白白错失了通过小厂实习实现曲线救国的机会。

因此，若你正身处中小厂或刚拿到此类 Offer，目前的重中之重是：

深入洞察业务逻辑，将平凡的工作进行结构化提炼与包装。

只有通过深挖产出价值，才能将其转化为通往大厂的有力敲门砖

那究竟怎么提炼包装才能提高自己面试的竞争力？

怎么提炼包装实习以提高自己面试的竞争力

这里给大家三个思考的方向

1.向上看（对齐okr）

- 翻okr文档/产品文档/技术设计文档，梳理自己组内做的东西用户是谁，痛点是什么
- 看老板今年最头疼什么（是成本、稳定性还是人效？）我做的内容，能不能跟这些挂钩
- 比如我在实习的时候，后端有个关键的指标，属于是比较重要但需要时间调研的活
 - 我主动学习了相关的业务，梳理了整个过程，跟我的mentor对齐后，给出了一个初版方案
 - 因为组里正职大家都忙于日常版本迭代的业务开发，因此我有机会作为一个实习生参与到一个

核心链路

- 有了参与的机会，自然而然我的日常开发需求会要求我去了解更完整的业务，对接上下游
- 这样让我的简历更好写同时，我在面试的时候，也能从业务到代码讲些内容

2.向下挖（极端场景）

- 正常的 CRUD 谁都会，但极端环境的处理和方案设计更能突出我们的能力
- 对于实习生来说，有些时候可能不会遇到这样的环境，但可以讲你有这么些设计考量
- 比如：大促流量翻倍怎么办？Kafka 堆积了怎么办？这些异常路径的处理才是竞争力的来源。

3.向宽走（跨团队）

- 去想你的用户是谁，过程中可能会跟哪些团队接触
- 比如你的链路涉及到上游接口的调用，那你可以顺着链路，剖析你这一环的作用
- 另一种视角是，一些大的故障出现，你可能会跟redis/kafka等团队接触，这些也是面试的时候可以讲的点

有了这三个思考的方向，就可以开始搜组里对应的文档，看对应的代码，请教自己的带教，最后总结成自己的话术

一个例子

假设我正在某厂的中台实习。Leader 给我的任务其实很普通：“做一个后台页面，能展示大家跑的任务现在的状态，是跑完了还是失败了。”

如果我只把它当 CRUD 做，那我简历上只能写：“使用 SpringBoot + MySQL 开发了任务状态查询接口。”

参考我先前给大家的三个思考的方向，我可以这么写：

向上看：

1. 我刚进组时，翻了组里的双周会纪要，发现老板和业务方（比如推荐组、运营组）经常吵架。
2. 我的思考：Leader 让我做的这个看板，除了看之外，还可以带预测和报警的功能
3. 最后写在简历上的，可以是我的这个思考加方案，带来了多少提升

向下挖：

1. 功能上线前，我自己在测试环境跑没问题，但肯定要考虑极端场景，如果在大促期间（比如双

11) , 任务量翻了 10 倍怎么办

2. 我自己设计了个方案, 做了写压测, 把问题同步给mentor, 迭代原先不合理的设计
3. 最终可以在简历上写自己对模块做了压测, 提高了稳定性

向宽走:

1. 系统做出来了, 但我发现大家还在扯皮。A 团队说 B 团队慢, B 说 C 慢。
2. 所以又可以迭代了—— 解决了跨部门定责的难题

那这么一分析, 最终我的简历上可以加这么几点:

- **负责核心业务链路的可视化与智能化升级。**针对下游业务方对数据产出时效性感知弱的痛点, 将基础的被动查询重构为**基于历史行为分析的智能预测模型**。该方案实现了对链路异常的主动预警, 将上下游协作效率提升 **XX%**, 有效支撑了 **XX 业务** 的日常决策制定。
- **主导高并发场景下的系统稳定性治理。**针对周期性流量洪峰可能引发的**中间件消息积压与扇出效应**, 独立设计压测方案并定位性能瓶颈。通过引入**多级缓存去重机制与流量分片策略**, 成功支撑 **XX 倍** 峰值负载下的状态变更冲击, 确保监控服务在极端场景下的**零延迟与高可用**。
- **构建分布式链路的自动化诊断与归因体系。**解决复杂依赖环境下“故障根因定位难、责任边界模糊”的治理难题。利用图算法技术, **实现对链路延迟的分钟级/秒级自动定位**, 为跨团队的服务质量考核提供了量化数据支撑, 推动了整体技术治理效率的提升。

校招大厂后端面试，到底怎么准备场景题

校招大厂后端面试，到底怎么准备场景题

写在前面：场景题是面试里最能拉开差距的部分，八股背得再熟，遇到开放性场景题还是会懵。这篇文章分享我准备场景题的方法，核心是借鉴北美System Design的思路来拓宽视野，虽然国内不考System Design，但这套思维方式对回答场景题帮助很大。

什么是场景题？为什么难？

场景题就是面试官给你一个实际业务场景，让你设计解决方案。比如：

- "如果你的系统突然流量暴增10倍，你怎么处理？"
- "设计一个秒杀系统，需要考虑哪些问题？"
- "用户反馈接口响应很慢，你会从哪些方面排查？"

为什么场景题难？

1. 没有标准答案：不像八股文有固定答案，场景题考察的是你的思维方式
2. 需要全局视角：不能只想到一个点，要从多个维度考虑（高并发、高可用、数据一致性等）
3. 考验实战经验：光背八股不够，得真正理解技术在实际场景中怎么用

我刚开始准备的时候也很头疼，看面经发现每个人的回答都不一样，不知道该怎么准备。后来摸索出一套方法，效果还不错。

我的准备方法：面经 + System Design

核心思路：看面经积累场景 + 看System Design学习思维框架

第一步：看面经积累常见场景

面经是必看的，但不是为了背答案，而是为了：

1. 了解高频场景：哪些场景经常被问到（秒杀、缓存、限流、降级等）

2. **看别人的思路**：同一个问题，不同人会从不同角度回答

3. **找到自己的盲区**：发现自己没想到的点

我当时主要看的是牛客、力扣的面经区，还有一些公众号（JavaGuide、小林coding等）。

看面经的技巧：

- 不要只看答案，要想"如果是我，我会怎么回答"
- 把高频场景整理出来，分类记录（缓存类、并发类、性能优化类等）
- 遇到不懂的技术点，马上去补八股

第二步：看System Design拓宽视野

这是我觉得最有用的部分。

System Design是什么？

System Design是北美大厂面试的一个环节，要求候选人在45分钟内设计一个大型系统（比如设计Twitter、设计Uber、设计YouTube）。虽然国内不考这个，但**这套思维方式对回答场景题帮助很大**。

为什么System Design对国内面试有用？

1. **提供了一套完整的思考框架**：从需求分析 → 容量估算 → 架构设计 → 技术选型 → 优化方案，这个流程可以直接套用到场景题
2. **训练全局思维**：System Design强调trade-off（权衡），让你学会从多个维度思考问题
3. **补充实战经验不足**：校招生项目经验有限，System Design里的案例可以补充你的"见识"

举个例子：

面试官问："设计一个短链接服务"

如果只看八股，你可能只想到"用哈希算法生成短链"。

但看过System Design后，你会想到：

- **需求澄清**：QPS多少？需要自定义短链吗？有效期多久？
- **容量估算**：每天生成多少短链？存储需要多大？
- **架构设计**：用什么数据库？如何保证高可用？
- **技术选型**：Redis做缓存，MySQL做持久化，布隆过滤器防止重复
- **优化方案**：如何应对热点短链？如何防止恶意刷接口？

这样回答，面试官会觉得你思路清晰，考虑全面。

我看的System Design资源

我主要看的是YouTube上的一个博主s09g，他在小红书上也有账号

强烈推荐这个博主

他的视频讲得非常好，每个案例都会从头到尾完整地走一遍设计流程，而且会讲清楚为什么这么设计，有哪些trade-off。

怎么看这些视频？

1. **不要全看**：视频很多，挑几个经典的看就行（比如设计URL Shortener、设计Rate Limiter、设计Cache）
2. **重点学框架**：不是记住具体设计，而是学习他的思考方式
3. **结合国内场景**：看完后想想，如果面试官问类似问题，我怎么表达出来

其他资源：

- Grokking the System Design Interview（付费课程，但网上有很多笔记）
- System Design Primer（GitHub上的开源项目，中文版也有）
- ByteByteGo（另一个很火的System Design频道，适合系统学习，我当时也买了书看）

我当时主要看的是s09g的视频，因为他讲得比较细，适合新手。

如何把System Design思路用到国内面试

看完System Design后，关键是要转化成自己的东西。

我的做法：

1. **整理思维框架**：把System Design的思考流程总结成自己的模板
2. **结合八股知识**：把学到的架构思路和国内八股（Redis、MySQL、MQ等）结合起来
3. **模拟练习**：看到面经里的场景题，用这套框架自己练习回答

我总结的场景题思考框架

这是我从System Design里提炼出来的，面试时可以直接套用：

第一步：明确问题

不要上来就回答，先问清楚：

- 业务规模多大？（QPS、用户量、数据量）
- 有哪些核心需求？（功能性需求）
- 有哪些非功能性需求？（高并发、高可用、低延迟）

第二步：拆解维度

从多个角度思考解决方案：

- **高并发**：如何应对大流量？（缓存、限流、降级）
- **高可用**：如何保证服务不挂？（主从、集群、熔断）
- **数据一致性**：如何保证数据正确？（事务、分布式锁、最终一致性）
- **性能优化**：如何提升响应速度？（索引、异步、预加载）

第三步：给出方案

针对每个维度，给出具体技术方案：

- 说清楚用什么技术（Redis、MQ、Nginx等）
- 说清楚为什么用这个技术（解决什么问题）
- 说清楚有什么trade-off（优缺点）

第四步：总结回顾

最后总结一下你的方案，让面试官记住你的思路。

实战案例：我面试中遇到的场景题

分享几个我实际面试中遇到的场景题，以及我是怎么用这套思路回答的。

案例1：秒杀系统设计（字节二面）

面试官问："设计一个秒杀系统，10万人抢100件商品，你会怎么做？"

我的回答思路：

1. 明确问题：

- 先确认QPS（假设10万人在1秒内请求，QPS=10万）
- 核心需求：库存扣减准确、不能超卖
- 非功能需求：高并发、低延迟

2. 拆解维度：

- 高并发：10万QPS，单机扛不住，需要分流
- 数据一致性：100件商品，不能超卖
- 用户体验：秒杀失败也要快速返回

3. 给出方案：

- 前端限流：按钮置灰、验证码、限制重复提交
- Nginx限流：漏桶算法，控制进入后端的流量
- Redis预减库存：把库存放Redis，用Lua脚本保证原子性
- 消息队列削峰：秒杀请求进MQ，后端慢慢消费
- 数据库兜底：最终扣减MySQL库存，用乐观锁防止超卖

4. 总结：

- 通过多层限流保证系统不被打垮
- 通过Redis+MQ保证高并发下的性能
- 通过乐观锁保证数据一致性

面试官追问："如果Redis挂了怎么办？"

我的回答：Redis做主从+哨兵，保证高可用。如果真的全挂了，降级到数据库，虽然慢但至少能用。

这个回答就是典型的System Design思路：先明确规模 → 拆解问题 → 给出方案 → 考虑边界情况。

案例2：接口响应慢排查（美团一面）

面试官问："用户反馈某个接口很慢，你会从哪些方面排查？"

我的回答思路：

我会从以下几个维度排查：

1. 应用层：

- 看日志，确认慢在哪个环节（数据库查询？外部调用？）
- 看代码，是否有循环查询、N+1问题

2. 数据库层：

- 看慢查询日志，是否有没加索引的查询
- 看执行计划，是否走了全表扫描
- 看数据量，是否需要分页或分表

3. 缓存层：

- 看缓存命中率，是否缓存失效了
- 看是否有缓存穿透、击穿问题

4. 网络层：

- 看是否有超时重试导致的雪崩

5. 系统资源：

- 看CPU、内存、磁盘IO是否打满
- 看是否有GC频繁导致的STW

面试官追问："如果是数据库慢查询，你会怎么优化？"

我的回答：

- 加索引（联合索引、覆盖索引）
- 优化SQL（避免select *、减少join）
- 如果数据量大，考虑分库分表
- 如果是热点数据，加Redis缓存

全局视角很重要，不是只想到一个点，而是从多个层面系统性地分析问题。

常见误区和注意事项

误区1：背System Design的具体设计

不要死记硬背"设计Twitter就用这个架构"，面试官不会问一模一样的问题。重要的是学习**思考方式**，而不是记住答案。

误区2：只看不练

看完视频要自己练习，找几个经典场景题（秒杀、短链接、限流、缓存）自己完整地设计一遍，最好能讲给别人听。

误区3：脱离国内实际

System Design里有些技术国内不常用（比如Cassandra、DynamoDB），面试时要转化成国内常用的技术栈（MySQL、Redis、RocketMQ等）。

误区4：忽略八股基础

System Design是锦上添花，不是雪中送炭。如果连Redis、MySQL的八股都不熟，先把基础打牢再看System Design。

给正在找实习/准备春招的同学们的建议 & 总结

作为校招生，我们的劣势是**项目经验不足**，很多场景没真正遇到过。但System Design可以帮我们：

1. **补充"见识"**：虽然没做过大型系统，但通过学习可以了解大厂是怎么设计的
2. **建立思维框架**：有了框架，即使遇到没见过的场景，也能有条理地分析
3. **提升表达能力**：System Design训练的是"把复杂问题讲清楚"的能力，这在面试中很重要

场景题是面试中最能体现思维能力的部分，准备方法就是：

1. **看面经**：积累常见场景，了解高频问题
2. **看System Design**：学习思维框架，拓宽视野
3. **多练习**：自己模拟回答，最好能讲给别人听

推荐资源：

- 小红书博主：[s09g](#)
- GitHub项目：System Design Primer
- 国内公众号：JavaGuide、小林coding

System Design不是为了背答案，而是为了训练思维方式。

当你能从多个维度系统性地分析问题时，场景题自然就不难了。

希望这篇文章对你有帮助，祝大家面试顺利！

叙述项目_场景题的回答模板

面试官抛了一个场景给你，不能想到哪回答到哪，我建议总分总的结构

1. 总：先思考你可能会回答的总体方案，然后告诉面试官你觉得针对这个场景，有A/B/C三种方案
 - a. ”面试官你好，针对刚才提及的场景，我从高并发/高可用/.....这X个角度给出一些解决措施“
2. 分：详细叙述各分点的内容，这个时候面试官可能会打断你提问，及时给出反应，不会的也要诚恳说确实没考虑过这方面 or 知识储备还没有这块，下来我会仔细研究一下，感谢您的提醒
 - a. ”对于高并发场景，我将采用D/E/F.....“
3. 总：最后总结，因为这个时候面试官聊了这么久，可能已经忘了你讲了什么了，所以需要你再总结一下，一是让面试官觉得你的表达是有层次，有逻辑的，二是方便面试官记面评，这很能刷好评
 - a. ”最后总结一下，我采用了A/B/C三种方案，其中A主要是.....，B主要是.....，C主要是“